Creating SELECT statements using one table

SELECT and FROM clause

```
SELECT *
FROM SalesLT.Product --; Only needed before CTE. Two part: schema and table
name

SELECT ProductID, Name, Color, ListPrice, SellStartDate
FROM [SalesLT].[Product];

SELECT ProductID,
Name,
Color,
ListPrice,
SellStartDate
FROM
[SalesLT].[Product]; -- You can have carriage returns/new lines - just not
in the middle of words.

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]

SELECT ProductID, Name, Color ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]

SELECT ProductID, Name Color, ListPrice, SellStartDate -- Problem
FROM [SalesLT].[Product]

SELECT Color AS [Product Color]
FROM [SalesLT].[Product]

SELECT Color AS [Product
Color]
FROM [SalesLT].[Product] -- Not advised.

SELECT DISTINCT Color AS ProductColor
FROM [SalesLT].[Product]

SELECT DISTINCT ProductCategoryID, ProductModelID
FROM SalesLT.Product;
```

WHERE clause – One condition

```
SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE ProductID = 710

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE ProductID > 710 -- What are the NULLs?
```



```
SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE ProductID >= 710 -- Make sure they are the right way round

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE ProductID < 710 -- What are the NULLs?

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE ProductID <= 710 -- Make sure they are the right way round

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE ProductID <> 710 – or !=

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE NOT (ProductID = 710) -- Brackets not essential, but can be useful.

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE Color = 'White' -- Cannot use " marks

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE Color = N'White'

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE ProductColor = N'White' -- Does not work. Cannot use alias in the
WHERE clause.

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE SellStartDate = '07/01/2005' – You can use 2 digits. This is based on
the two digit year cutoff.

-- SELECT * FROM sys.configurations – look for "two digit year cutoff"

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE SellStartDate = '07-01-2005' – However, the default dateformat can be
changed using SET DATEFORMAT dmy;

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE SellStartDate = 'JUL 01 2005' -- Other formats are available.

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE SellStartDate = '2005-07-01'
```



```
SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE SellStartDate = '20050701' -- ISO 8601 format

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE SellStartDate = '2005-07-01 00:00:00' -- ISO 8601 format

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE SellStartDate = '2005-07-01T00:00:00' -- ISO 8601 format
```

WHERE clause – LIKE

```
SELECT Name
FROM [SalesLT].[Product]

SELECT Name
FROM [SalesLT].[Product]
WHERE Name = 'LL Mountain Frame' -- Gives no results

SELECT Name
FROM [SalesLT].[Product]
WHERE Name = 'LL Mountain Frame*' -- Gives no results

SELECT Name
FROM [SalesLT].[Product]
WHERE Name LIKE 'LL Mountain Frame' -- Does not work

SELECT Name
FROM [SalesLT].[Product]
WHERE Name LIKE 'LL Mountain Frame*' -- Also does not work

SELECT Name
FROM [SalesLT].[Product]
WHERE Name LIKE 'LL Mountain Frame%' -- Does work

SELECT Name
FROM [SalesLT].[Product]
WHERE Name LIKE '%Mountain Frame%'

SELECT Name
FROM [SalesLT].[Product]
WHERE Name LIKE '%38'

SELECT Name
FROM [SalesLT].[Product]
WHERE Name LIKE '_L Mountain Frame%38'

SELECT Name AS ProductName
FROM SalesLT.Product
WHERE Name LIKE '_L Mountain Frame%Black%'
```



```
SELECT DISTINCT Color
FROM [SalesLT].[Product]
WHERE Color LIKE 'b%' COLLATE SQL_Latin1_General_CP1_CS_AS -- Case
sensitive search. Can also use _CI for case insensitive.

SELECT DISTINCT Color
FROM [SalesLT].[Product]
WHERE Color LIKE '[BM]%' -- Searching for B or M

SELECT DISTINCT Color
FROM [SalesLT].[Product]
WHERE Color LIKE '[B-M]%' -- Searching for B to M

SELECT DISTINCT Color
FROM [SalesLT].[Product]
WHERE Color LIKE '[^B-M]%' -- Searching for other than B to M

SELECT DISTINCT ThumbnailPhotoFileName
FROM [SalesLT].[Product]
WHERE ThumbnailPhotoFileName LIKE '%image[_]available%'

SELECT DISTINCT ThumbnailPhotoFileName
FROM [SalesLT].[Product]
WHERE ThumbnailPhotoFileName LIKE '%image[_]available%'
```

WHERE clause – Multiple conditions

```
SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE Color = 'White' OR ProductID < 710

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE Color = 'White' OR Color = 'Multi'

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE Color IN ('White', 'Multi')

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE ProductID >=710 AND ProductID <= 719 -- Can use brackets

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE ProductID >=710 AND ProductID < 720 --May give a different result, if
you have a ProductID 719.5

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE ProductID BETWEEN 710 AND 719 -- Not "IS BETWEEN"
```



```
SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE ProductID NOT BETWEEN 710 AND 719

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE NOT ProductID BETWEEN 710 AND 719
```

WHERE clause – NULL

```
SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product] -- 295 rows

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE Color = 'White' -- 4 rows

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE Color <> 'White' -- 241 rows. Where are the other 50 rows?

-- Given Name: 'John'
-- Middle Name(s): NULL
-- Family Name: 'Smith'

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE Color = NULL -- does not work.

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE Color IS NULL -- 50 rows.

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE Color IS NOT NULL -- 245 rows. You can also use WHERE NOT Color IS
NULL

SELECT ProductID, Name, Color AS ProductColor, ListPrice, SellStartDate
FROM [SalesLT].[Product]
WHERE Color NOT IN ('White', 'Multi') OR Color IS NULL
```

Aggregations and the GROUP BY clause

```
SELECT *
FROM [SalesLT].[Product]

SELECT COUNT(*)
FROM [SalesLT].[Product] -- All rows

SELECT COUNT(Weight)
FROM [SalesLT].[Product] -- Only non-NULL values
```



```
SELECT COUNT(*)
FROM [SalesLT].[Product]
WHERE Weight IS NOT NULL

SELECT COUNT(*)
FROM [SalesLT].[Product]
WHERE Weight IS NULL

SELECT COUNT(*) AS NumberOfRows, COUNT(Weight) AS NonNullRows, SUM(Weight)
AS WeightSum
     , MIN(Weight) AS WeightMin, MAX(Weight) AS WeightMax
FROM SalesLT.Product

SELECT COUNT(*) AS NumberOfRows, COUNT(Name) AS NonNullRows
     , MIN(Name) AS WeightMin, MAX(Name) AS WeightMax
FROM SalesLT.Product
-- SUM can only be used for numeric data

SELECT Color AS ProductColor, COUNT(*) AS NumberOfRows
FROM [SalesLT].[Product] -- Does not work

SELECT Color, COUNT(*) AS NumberOfRows
FROM [SalesLT].[Product]
GROUP BY Color -- When using aggregated AND non-aggregated data, you must
use the GROUP BY clause.

SELECT Color AS ProductColor, COUNT(*) AS NumberOfRows
FROM [SalesLT].[Product]
GROUP BY ProductColor -- Cannot use alias in the GROUP BY clause.

SELECT ProductCategoryID, Color, COUNT(*) AS NumberOfRows
FROM [SalesLT].[Product]
GROUP BY Color -- Does not work

SELECT ProductCategoryID, Color, COUNT(*) AS NumberOfRows
FROM [SalesLT].[Product]
GROUP BY Color, ProductCategoryID -- Can be in either order
```

HAVING clause

```
SELECT Color, COUNT(*) AS NumberOfRows
FROM [SalesLT].[Product]
GROUP BY Color

SELECT Color, COUNT(*) AS NumberOfRows
FROM [SalesLT].[Product]
WHERE NumberOfRows>=30 -- Does not work
GROUP BY Color

SELECT Color, COUNT(*) AS NumberOfRows
FROM [SalesLT].[Product]
WHERE COUNT(*)>=30 -- Does not work
```



```
GROUP BY Color

SELECT Color, COUNT(*) AS NumberOfRows
FROM [SalesLT].[Product]
GROUP BY Color
WHERE COUNT(*)>=30 -- Does not work

SELECT Color, COUNT(*) AS NumberOfRows
FROM [SalesLT].[Product]
GROUP BY Color
HAVING COUNT(*)>=30 -- Works!

SELECT Color, COUNT(*) AS NumberOfRows
FROM [SalesLT].[Product]
GROUP BY Color
HAVING NumberOfRows>=30 -- Does not Work

SELECT Color, COUNT(*) AS NumberOfRows
FROM [SalesLT].[Product]
GROUP BY Color
HAVING Color = 'Red' -- Does work, but is not usual.
```

ORDER BY clause

```
SELECT *
FROM [SalesLT].[Product]
ORDER BY Name

SELECT *
FROM [SalesLT].[Product]
ORDER BY Name DESC

SELECT *
FROM [SalesLT].[Product]
ORDER BY Name ASC

SELECT *
FROM [SalesLT].[Product]
ORDER BY ProductCategoryID -- non-deterministic

SELECT *
FROM [SalesLT].[Product]
ORDER BY ProductCategoryID, ProductModelID, ProductID

SELECT *
FROM [SalesLT].[Product]
ORDER BY ProductCategoryID, ProductModelID, ProductID DESC

SELECT *
FROM [SalesLT].[Product]
ORDER BY ProductCategoryID ASC, ProductModelID ASC, ProductID DESC

SELECT *
```



```
FROM [SalesLT].[Product]
ORDER BY ProductCategoryID DESC, ProductModelID, ProductID

SELECT *
FROM [SalesLT].[Product]
ORDER BY 5 ASC, 1 DESC -- You can order by column number. This is not
recommended.

SELECT DISTINCT Weight
FROM [SalesLT].[Product]
ORDER BY Weight -- The NULL is sorted first

SELECT DISTINCT Weight
FROM [SalesLT].[Product]
ORDER BY Weight DESC -- The NULL is sorted last

SELECT DISTINCT TOP 10 Weight
FROM [SalesLT].[Product]
ORDER BY Weight

SELECT DISTINCT TOP (10) Weight
FROM [SalesLT].[Product]
ORDER BY Weight

SELECT DISTINCT TOP (10) Weight AS WeightKg
FROM SalesLT.Product
ORDER BY WeightKg
```

Example using all 6 clauses

```
SELECT Color, COUNT(*) AS NumberOfRows
FROM [SalesLT].[Product]
WHERE ProductID > 800 -- AND Color IS NOT NULL
GROUP BY Color
HAVING COUNT(*) > 20
ORDER BY NumberOfRows DESC
```

Design and implement database objects

1a-1b. Design and implement tables, including data types and size

Number data types

```
-- Numbers without decimal places

-- bit 0 means false, bit 1 means true. Up to 8 bits in 1 byte.
-- tinyint is an unsigned integer that can store values from 0 to 255. It
uses 1 byte of storage.
-- smallint is a signed integer that can store values from -32,768 to
32,767. It uses 2 bytes of storage.
-- int is a signed integer that can store values from -2,147,483,648 to
2,147,483,647. It uses 4 bytes of storage.
```



```
-- bigint is a signed integer that can store values from -
9,223,372,036,854,775,808 to 9,223,372,036,854,775,807. It uses 8 bytes of
storage.

DECLARE @myVar AS INT = 4
SELECT @myVar as myVariable

SET @myVar = @myVar + 1
SELECT @myVar as myResult -- + 2, -, *, % (modulo), / (division)
GO

-- Numbers with decimal places

DECLARE @myVar AS DECIMAL(18, 9) = 123456789.123456789 -- or NUMERIC -
START AT default, then (18, 9)
SELECT @myVar as myVariable
GO

-- smallmoney is a currency value that can store values from -214,748.3648
to 214,748.3647. It uses 4 bytes of storage.
-- money is a currency value that can store values from -
922,337,203,685,477.5808 to 922,337,203,685,477.5807. It uses 8 bytes of
storage.

DECLARE @myVar AS smallmoney = 123456.7891
SELECT @myVar as myVariable
GO

DECLARE @myVar AS money = 123456789.7891
SELECT @myVar as myVariable
GO

-- Approximate numeric data types: float and real. They are used to store
approximate values with floating-point precision.
DECLARE @myVar AS float(24) = 1234.56789
SELECT @myVar as myVariable
GO

DECLARE @myVar AS float(53) = 1234.56789
SELECT @myVar as myVariable
```

Functions dealing with NULL

```
-- Numerical operations involving NULL values will return NULL. Use
COALESCE or IIF to handle NULL values.
DECLARE @myVar AS INT -- Change: add = 2
SELECT @myVar + 2 as myResult -- NULL if any operand is NULL
SELECT COALESCE(@myVar + 2, 0) as myResult -- 0 if any operand is NULL
SELECT IIF(@myVar IS NULL, 0, @myVar + 2) as myResult -- NULL if any
operand is NULL
SELECT CASE WHEN @myVar IS NULL THEN 0 ELSE @myVar + 2 END as myResult --
NULL if any operand is NULL
```



```
SELECT CASE @myVar WHEN NULL THEN 2 ELSE @myVar + 2 END as myResult -- Does
not work
```

String data types of functions

```
-- Strings

DECLARE @MyVar NVARCHAR(max) = 'Hello' -- Start with CHAR(20), Change to
VARCHAR, add , then NVARCHAR, then N'', then (max)


SELECT @MyVar AS myChar, LEN(@MyVar) AS CharLength, DATALENGTH(@MyVar) AS
DataLength


-- Do not use text and ntext data types, as they are deprecated. Use
varchar(max) and nvarchar(max) instead.

GO

-- String functions

DECLARE @MyVar NVARCHAR(20) = 'Hello'


SELECT @MyVar + ' World' AS ConcatenatedString, -- Concatenation

         UPPER(@MyVar) AS UpperCase, -- Convert to uppercase

         LOWER(@MyVar) AS LowerCase, -- Convert to lowercase

         LEFT(@MyVar, 2) AS LeftPart, -- Get left part of the string

         RIGHT(@MyVar, 2) AS RightPart, -- Get right part of the string

         SUBSTRING(@MyVar, 2, 3) AS SubString, -- Get substring starting
from position 2 with length 3

         LEN(@MyVar) AS StringLength, -- Get length of the string

         DATALENGTH(@MyVar) AS DataLength, -- Get data length in bytes

         @MyVar + NULL AS NULLString,

         TRIM('   Hello   World  ') As TrimmedString,

         LTRIM('   Hello   World  ') As LTrimmedString,

         RTRIM('   Hello   World  ') As RTrimmedString

GO


DECLARE @MyString NVARCHAR(20) = 'Hello'

DECLARE @MyNumber INT = 1

SELECT @MyString + @MyNumber AS ConcatenatedWithNumber -- Does not work.

SELECT @MyString + 1 AS ConcatenatedWithString --Change 1 to '1'
```



```
SELECT @MyString + CAST('1' AS NVARCHAR(20)) AS ConcatenatedWithString --
Works

SELECT @MyString + CONVERT(NVARCHAR(20), '1') AS ConcatenatedWithString --
Works

GO


-- Converting to number

DECLARE @MyString NVARCHAR(20) = 'hello'

SELECT 1 + 1.234 AS PlusNumber -- Works

-- SELECT 1 + @MyString AS PlusNumber -- Does not work.

-- SELECT 1 + CAST(@MyString AS DECIMAL(9,5)) AS PlusNumber -- work.

SELECT 1 + TRY_CAST(@MyString AS DECIMAL(9,5)) AS PlusNumber

GO


-- Formatting numbers

DECLARE @MyString NVARCHAR(20) = 'The price is: '

DECLARE @MyNumber DECIMAL(9,2) = 1234.50

SELECT @MyString + '$' + CAST(@MyNumber AS NVARCHAR(20)) AS Price

SELECT @MyString + FORMAT(@MyNumber, 'C', 'en-US') AS Price
```

Date data type and functions

```
DECLARE @myDate as DATETIME2(7) = '2032-02-03T01:02:03.1234567' -- change
from date to time, smalldatetime, datetime, datetime2

SELECT @myDate as DateValue, DATALENGTH(@myDate) as DataLength,
SQL_VARIANT_PROPERTY(@myDate, 'BaseType') as BaseType

GO


DECLARE @myDate as DATETIMEOFFSET = '2032-02-03 01:02:03.1234567 -05:00' --
cannot use T format for datetimeoffset, must use space between date and
time, and include offset

SELECT @myDate as DateValue, DATALENGTH(@myDate) as DataLength,
SQL_VARIANT_PROPERTY(@myDate, 'BaseType') as BaseType

GO


-- Manipulating dates

DECLARE @myDate as DATETIME2(7) = '2032-02-03T01:02:03.1234567'
```



```
SELECT DATEADD(YEAR, 1, @myDate) as AddYear, DATEADD(MONTH, -1, @myDate) as
AddMonth , DATEADD(DAY, 10, @myDate) as AddDay, DATEADD(HOUR, 5, @myDate)
as AddHour,

        DATEADD(MINUTE, -30, @myDate) as AddMinute, DATEADD(SECOND, 15,
@myDate) as AddSecond,

        DATEADD(MILLISECOND, 500, @myDate) as AddMillisecond

GO


-- What is the current date/time?

SELECT GETDATE() AS GetDate, SYSDATETIME() AS SysDateTime

     , SYSUTCDATETIME() AS SysUTC, SYSDATETIMEOFFSET() AS SysDateTimeOffset


-- Converting dates to numbers

DECLARE @myDate as DATETIME2(7) = '2032-02-03T01:02:03.1234567'

SELECT YEAR(@myDate) as YearValue, MONTH(@myDate) as MonthValue,
DAY(@myDate) as DayValue

GO


DECLARE @myDate as DATETIME2(7) = '2032-02-03T01:02:03.1234567'

SELECT DATEDIFF(MONTH, @myDate2, @myDate) as DaysDifference

SELECT DATEPART(YEAR, @myDate) as QuarterValue, DATEPART(MONTH, @myDate) as
WeekDayValue

     , DATEPART(DAY, @myDate) as DayValue, DATEPART(HOUR, @myDate) as
HourValue

     , DATEPART(MINUTE, @myDate) as MinuteValue2, DATEPART(SECOND, @myDate)
as SecondValue2

     , DATEPART(MILLISECOND, @myDate) as MillisecondValue,
DATEPART(MICROSECOND, @myDate) as MicrosecondValue

     , DATEPART(NANOSECOND, @myDate) as NanosecondValue

SELECT DATEPART(QUARTER, @myDate) as QuarterValue, DATEPART(WEEKDAY,
@myDate) as WeekDayValue

     , DATEPART(DAYOFYEAR, @myDate) as DayOfYearValue, DATEPART(WEEK,
@myDate) as WeekValue

GO


-- Converting dates to strings

DECLARE @myDate as DATETIME2(7) = '2032-02-03T01:02:03.1234567'
```



```
SELECT DATENAME(WEEKDAY, @myDate) as WeekDayName, DATENAME(MONTH, @myDate)
as MonthName

SELECT FORMAT(@myDate, 'dddd d MMMM yyyy HH:mm:ss.fffffff') as
FormattedDate

SELECT FORMAT(@myDate, 'D') as FormattedDate -- Long date format

SELECT FORMAT(@myDate, 'd') as FormattedDate -- Short date format

SELECT FORMAT(@myDate, 'D', 'es-ES') as FormattedDateSpanish -- Long date
format

GO


-- Converting strings to dates

DECLARE @myString as NVARCHAR(30) = '2032-02-03 01:02:03.1234567'

SELECT CAST(@myString AS DATETIME2(7)) as CastDate, TRY_CAST(@myString AS
DATETIME2(7)) as TryCastDate

GO


DECLARE @myString as NVARCHAR(30) = 'Tuesday, 3 February 2032' -- change
from Tuesday, 3 February 2032 to martes, 3 febrero 2032

SELECT PARSE(@myString AS DATETIME2(7) USING 'es-US') as ParseDate,
TRY_PARSE(@myString AS DATETIME2(7) USING 'es-US') as TryParseDate --
change to es-US


DECLARE @myDate as DATETIME2(7) = '2032-02-03T01:02:03.1234567'

DECLARE @myDate2 as DATETIME2(7) = '2032-01-01'

SELECT DATEDIFF(MONTH, @myDate2, @myDate) as DaysDifference
```

1c. Design and implement tables

Creating tables

```
CREATE SCHEMA School
GO

CREATE TABLE School.Scores
(
      PupilID INT NOT NULL,
    Score SMALLINT NOT NULL,
      SetType CHAR(1)
)

DROP TABLE School.Scores;

ALTER TABLE School.Scores
```



```
ALTER COLUMN SetType VARCHAR(10) NOT NULL;

ALTER TABLE School.Scores
ADD DateTaken DATE NULL;

ALTER TABLE School.Scores
DROP COLUMN DateTaken;

-- Data Definition Language (DDL) statements are used to define and modify
database structures, such as tables, schemas, and columns. In the provided
code snippet, we see a series of DDL statements that create a new schema
called "School," create a table named "Scores" within that schema, and then
perform various alterations on the table structure.
```

Inserting, Updating and Deleting Data

```
ALTER TABLE School.Scores
ADD DateTaken DATE NULL;

SELECT * FROM [School].[Scores]

INSERT INTO [School].[Scores] (PupilID, SetType, Score)
VALUES (1, 'Math', 85),
         (2, 'Math', 90),
         (3, 'Math', 78),
         (4, 'Science', 92),
         (5, 'Science', 88),
         (6, 'Science', 80);

UPDATE [School].[Scores]
SET DateTaken = '2032-10-15';

INSERT INTO [School].[Scores] (PupilID, SetType, Score, DateTaken)
VALUES (1, 'Math', 85, '2032-10-01'),
       (2, 'Math', 90, '2032-10-01'),
       (3, 'Math', 78, '2032-10-01'),
       (4, 'Science', 92, '2032-10-01'),
       (5, 'Science', 88, '2032-10-01'),
       (6, 'Science', 80, '2032-10-01'),
       (4, 'Math', 70, '2032-11-02'),
       (5, 'Math',76, '2032-11-02'),
       (7, 'Math',91, '2032-11-02'),
       (1, 'Science',42, '2032-11-02'),
       (2, 'Science',18, '2032-11-02'),
       (3, 'Science',80, '2032-11-02');

DELETE FROM [School].[Scores]
FROM [School].[Scores]
-- WHERE DateTaken IS NULL;
WHERE DateTaken = '2032-10-15'

SELECT @@ROWCOUNT AS RowsDeleted;
```



```
-- DDL = Data Definition Language
-- DML = Data Manipulation Language
```

Alternative ways of creating tables, inserting and deleting data

```
SELECT *
INTO #Scores2
FROM [School].[Scores]
WHERE DateTaken = '2032-10-01';

SELECT * FROM #Scores2;

SELECT *
INTO #Scores3
FROM [School].[Scores]
WHERE DateTaken = '2032-11-01';

UPDATE #Scores2
SET DateTaken = '2032-10-15'
OUTPUT deleted.*
INTO #Scores3;

DELETE FROM #Scores3;

TRUNCATE TABLE #Scores2;
```

4. Design and implement database constraints

4c. Unique Constraints

```
CREATE UNIQUE CLUSTERED INDEX UQ_Scores ON [School].[Scores](PupilID,
SetType, DateTaken);

ALTER TABLE [School].[Scores4]
ADD CONSTRAINT UQ_Scores4 UNIQUE NONCLUSTERED (PupilID, SetType,
DateTaken);

INSERT INTO [School].[Scores2] (PupilID, Score, SetType, DateTaken)
VALUES (1, 90, 'Math', '2032-10-02');

DROP TABLE IF EXISTS [School].[Scores3]
GO

DROP TABLE IF EXISTS [School].[Scores4]
GO

CREATE TABLE [School].[Scores3](
      [PupilID] [int] NOT NULL,
      [Score] [smallint] NOT NULL,
      [SetType] [varchar](10) NOT NULL,
      [DateTaken] [date] NULL,
      CONSTRAINT UX_Scores3 UNIQUE NONCLUSTERED (PupilID, SetType,
DateTaken)
) ON [PRIMARY]
```



```
GO

CREATE TABLE [School].[Scores4](
      [PupilID] [int] NOT NULL,
      [Score] [smallint] NOT NULL,
      [SetType] [varchar](10) NOT NULL,
      [DateTaken] [date] NULL UNIQUE NONCLUSTERED
) ON [PRIMARY]
GO

DROP TABLE IF EXISTS [School].[Scores4]
GO

CREATE TABLE [School].[Scores4](
      [PupilID] [int] NOT NULL,
      [Score] [smallint] NOT NULL,
      [SetType] [varchar](10) NOT NULL,
      [DateTaken] [date] NULL CONSTRAINT UX_Scores4 UNIQUE NONCLUSTERED
) ON [PRIMARY]
GO
```

4d. Check Constraint

```
ALTER TABLE [School].[Scores2]
ADD CONSTRAINT ck_Scores2 CHECK (Score BETWEEN 0 AND 100);

INSERT INTO [School].[Scores2] (PupilID, Score, SetType, DateTaken)
VALUES (1, 190, 'Math', '2032-10-03'); -- and -190.

DROP TABLE IF EXISTS [School].[Scores3]
GO

DROP TABLE IF EXISTS [School].[Scores4]
GO

CREATE TABLE [School].[Scores3](
      [PupilID] [int] NOT NULL,
      [Score] [smallint] NOT NULL,
      [SetType] [varchar](10) NOT NULL,
      [DateTaken] [date] NULL,
      CONSTRAINT UQ_Scores3 UNIQUE NONCLUSTERED (PupilID, SetType,
DateTaken),
      CONSTRAINT CK_Scores3_Score CHECK (Score BETWEEN 0 AND 100)
)
GO

CREATE TABLE [School].[Scores4](
      [PupilID] [int] NOT NULL,
      [Score] [smallint] NOT NULL CHECK (Score BETWEEN 0 AND 100),
      [SetType] [varchar](10) NOT NULL,
      [DateTaken] [date] NULL UNIQUE NONCLUSTERED
)
```



```
GO
```

4a. Primary Key Constraint

```
DROP TABLE IF EXISTS School.Scores3
GO

CREATE TABLE School.Scores3(
      PupilID int NOT NULL,
      Score smallint NOT NULL,
      SetType varchar(10) NOT NULL,
      DateTaken date NULL,
      ScoresID int NOT NULL,
)
GO

ALTER TABLE School.Scores3
ADD CONSTRAINT PK_Scores3 PRIMARY KEY CLUSTERED (ScoresID)
GO

INSERT INTO School.Scores3(PupilID, SetType, Score, DateTaken, ScoresID)
VALUES (1, 'Math', 85, '2032-10-01', 1),
       (2, 'Math', 90, '2032-10-01', 2),
       (3, 'Math', 78, '2032-10-01', 3),
       (4, 'Science', 92, '2032-10-01', 4),
       (5, 'Science', 88, '2032-10-01', 5),
       (6, 'Science', 80, '2032-10-01', 6),
       (4, 'Math', 70, '2032-11-02', 7),
       (5, 'Math',76, '2032-11-02', 8),
       (7, 'Math',91, '2032-11-02', 9),
       (1, 'Science',42, '2032-11-02', 10),
       (2, 'Science',18, '2032-11-02', 11),
       (3, 'Science',80, '2032-11-02', 12);

INSERT INTO School.Scores3(PupilID, SetType, Score, DateTaken, ScoresID)
VALUES (1, 'Math', 85, '2032-10-01', 11) -- This does not work
---

DROP TABLE IF EXISTS School.Scores3
GO

CREATE TABLE School.Scores3(
      PupilID int NOT NULL,
      Score smallint NOT NULL,
      SetType varchar(10) NOT NULL,
      DateTaken date NULL,
      ScoresID int NOT NULL PRIMARY KEY,
)
GO
---

DROP TABLE IF EXISTS School.Scores3
```



```
GO

CREATE TABLE School.Scores3(
      PupilID int NOT NULL,
      Score smallint NOT NULL,
      SetType varchar(10) NOT NULL,
      DateTaken date NULL,
      ScoresID int,
      CONSTRAINT PK_Scores3 PRIMARY KEY (ScoresID)
)
GO

--

INSERT INTO School.Scores3(PupilID, SetType, Score, DateTaken)
VALUES (8, 'Math', 86, '2032-10-01')
SELECT * FROM School.Scores3;
```

4e. Default Constraint

```
DROP TABLE IF EXISTS School.Scores3
GO

CREATE TABLE School.Scores3(
      PupilID int NOT NULL,
      Score smallint NOT NULL,
      SetType varchar(10) NOT NULL,
      DateTaken date NULL,
      ScoresID int NOT NULL PRIMARY KEY,
      DateRecorded datetime2 NULL
)
GO

ALTER TABLE School.Scores3
ADD CONSTRAINT DF_DateRecorded DEFAULT GETDATE() FOR DateRecorded
GO

INSERT INTO School.Scores3(PupilID, SetType, Score, DateTaken, ScoresID)
VALUES (1, 'Math', 85, '2032-10-01', 1),
         (2, 'Math', 90, '2032-10-01', 2),
         (3, 'Math', 78, '2032-10-01', 3),
         (4, 'Science', 92, '2032-10-01', 4),
         (5, 'Science', 88, '2032-10-01', 5),
         (6, 'Science', 80, '2032-10-01', 6),
       (4, 'Math', 70, '2032-11-02', 7),
       (5, 'Math',76, '2032-11-02', 8),
         (7, 'Math',91, '2032-11-02', 9),
       (1, 'Science',42, '2032-11-02', 10),
       (2, 'Science',18, '2032-11-02', 11),
       (3, 'Science',80, '2032-11-02', 12);

SELECT * FROM School.Scores3
```



```
---

DROP TABLE IF EXISTS School.Scores3
GO

CREATE TABLE School.Scores3(
      PupilID int NOT NULL,
      Score smallint NOT NULL,
      SetType varchar(10) NOT NULL,
      DateTaken date NULL,
      ScoresID int NOT NULL PRIMARY KEY IDENTITY,
      DateRecorded datetime2 NULL DEFAULT GETDATE()
)
GO

SET IDENTITY_INSERT School.Scores3 OFF

INSERT INTO School.Scores3(PupilID, SetType, Score, DateTaken)
VALUES (1, 'Math', 85, '2032-10-01'),
         (2, 'Math', 90, '2032-10-01'),
         (3, 'Math', 78, '2032-10-01'),
         (4, 'Science', 92, '2032-10-01'),
         (5, 'Science', 88, '2032-10-01'),
         (6, 'Science', 80, '2032-10-01'),
       (4, 'Math', 70, '2032-11-02'),
       (5, 'Math',76, '2032-11-02'),
         (7, 'Math',91, '2032-11-02'),
       (1, 'Science',42, '2032-11-02'),
       (2, 'Science',18, '2032-11-02'),
       (3, 'Science',80, '2032-11-02');

SELECT * FROM School.Scores3;
---
SELECT * FROM School.Scores

ALTER TABLE School.Scores
ADD DateRecorded datetime2 NULL DEFAULT GETDATE()

ALTER TABLE School.Scores
ADD ScoresID int NOT NULL PRIMARY KEY IDENTITY
```

5. Design and implement SEQUENCEs

```
DROP TABLE IF EXISTS School.Scores3
GO

DROP SEQUENCE IF EXISTS seqScoresID
GO

CREATE SEQUENCE seqScoresID AS int
START WITH 1
INCREMENT BY 1
MINVALUE 1
```



```
MAXVALUE 3
CYCLE
CACHE 50
GO

CREATE TABLE School.Scores3(
    PupilID int NOT NULL,
    Score smallint NOT NULL,
    SetType varchar(10) NOT NULL,
    DateTaken date NULL,
    ScoresID int NULL CONSTRAINT DF_ScoresID DEFAULT NEXT VALUE FOR
seqScoresID,
    DateRecorded datetime2 NULL DEFAULT GETDATE()
)
GO

CREATE TABLE School.Scores3(
    PupilID int NOT NULL,
    Score smallint NOT NULL,
    SetType varchar(10) NOT NULL,
    DateTaken date NULL,
    ScoresID int NULL,
    DateRecorded datetime2 NULL DEFAULT GETDATE()
)
GO

ALTER TABLE School.Scores3
ADD CONSTRAINT DF_ScoresID DEFAULT NEXT VALUE FOR seqScoresID FOR ScoresID
GO

INSERT INTO School.Scores3(PupilID, SetType, Score, DateTaken)
VALUES
(1, 'Math', 85, '2032-10-01'),
(2, 'Math', 90, '2032-10-01'),
(3, 'Math', 78, '2032-10-01'),
(4, 'Science', 92, '2032-10-01'),
(5, 'Science', 88, '2032-10-01'),
(6, 'Science', 80, '2032-10-01'),
(4, 'Math', 70, '2032-11-02'),
(5, 'Math', 76, '2032-11-02'),
(7, 'Math', 91, '2032-11-02'),
(1, 'Science', 42, '2032-11-02'),
(2, 'Science', 18, '2032-11-02'),
(3, 'Science', 80, '2032-11-02')
GO

SELECT * FROM School.Scores3;

SELECT * FROM sys.sequences;

UPDATE School.Scores3
SET ScoresID = NEXT VALUE FOR seqScoresID
```


Creating SELECT statements using more than one table

Creating our first join

```
DROP TABLE IF EXISTS School.Scores2
DROP TABLE IF EXISTS School.Scores3
DROP TABLE IF EXISTS School.ScoresCopy
DROP TABLE IF EXISTS School.Pupils
CREATE TABLE School.Pupils
(
    PupilID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    ClassID INT
);

INSERT INTO School.Pupils (PupilID, FirstName, LastName, ClassID)
VALUES
(1, 'Alice', 'Jones', 1),
(2, 'Bob', 'Taylor', 1),
(3, 'Charlie', 'Evans', 1),
(4, 'Daisy', 'Wilson', 2),
(5, 'Ethan', 'Thomas', 2),
(8, 'Fiona', 'Roberts', 3);

SELECT * FROM School.Scores;
SELECT * FROM School.Pupils;

SELECT S.*, P.PupilID, P.FirstName, P.LastName
FROM School.Scores AS S
JOIN School.Pupils AS P
ON S.PupilID = P.PupilID;
```

Different types of joins

```
SELECT * FROM School.Scores
SELECT * FROM School.Pupils

-- An INNER JOIN
SELECT *
FROM School.Scores AS S
JOIN School.Pupils AS P
ON S.PupilID = P.PupilID;

-- A LEFT JOIN
SELECT *
FROM School.Scores AS S
LEFT JOIN School.Pupils AS P
ON S.PupilID = P.PupilID;

-- A RIGHT JOIN
SELECT *
```



```
FROM School.Scores AS S
RIGHT JOIN School.Pupils AS P
ON S.PupilID = P.PupilID;

-- Creating a single column for PupilID using a FULL JOIN.
SELECT ISNULL(S.PupilID, P.PupilID) AS PupilID,
       S.Score, S.SetType, S.DateTaken, S.Maximum,
       P.PupilID, P.FirstName, P.LastName
FROM School.Scores AS S
FULL JOIN School.Pupils AS P
ON S.PupilID = P.PupilID;

-- Searching for missing data in the Pupils table
SELECT S.*, P.PupilID, P.FirstName, P.LastName
FROM School.Scores AS S
LEFT JOIN School.Pupils AS P
ON S.PupilID = P.PupilID
WHERE P.PupilID IS NULL;

-- Searching for missing data in the Scores table
SELECT S.*, P.PupilID, P.FirstName, P.LastName
FROM School.Scores AS S
RIGHT JOIN School.Pupils AS P
ON S.PupilID = P.PupilID
WHERE S.PupilID IS NULL;
```

4b. Creating a FOREIGN KEY constraint

Creating tables

```
DROP TABLE IF EXISTS School.ScoresCopy
DROP TABLE IF EXISTS School.PupilsCopy

CREATE TABLE School.ScoresCopy(
    PupilID int NULL,
    Score smallint NOT NULL,
    SetType varchar(10) NOT NULL,
    DateTaken date NULL,
    ScoresID int NULL,
    DateRecorded datetime2 NULL DEFAULT GETDATE()
)
GO

CREATE TABLE School.PupilsCopy
(
    PupilID INT , --PRIMARY KEY
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    ClassID INT
);

INSERT INTO School.ScoresCopy(PupilID, SetType, Score, DateTaken)
VALUES
```



```
(1, 'Math', 85, '2032-10-01'),
(2, 'Math', 90, '2032-10-01'),
(3, 'Math', 78, '2032-10-01'),
(4, 'Science', 92, '2032-10-01'),
(5, 'Science', 88, '2032-10-01'),
(6, 'Science', 80, '2032-10-01'),
(4, 'Math', 70, '2032-11-02'),
(5, 'Math', 76, '2032-11-02'),
(7, 'Math', 91, '2032-11-02'),
(1, 'Science', 42, '2032-11-02'),
(2, 'Science', 18, '2032-11-02'),
(3, 'Science', 80, '2032-11-02')
GO

INSERT INTO School.PupilsCopy (PupilID, FirstName, LastName, ClassID)
VALUES
(1, 'Alice', 'Jones', 1),
(2, 'Bob', 'Taylor', 1),
(3, 'Charlie', 'Evans', 1),
(4, 'Daisy', 'Wilson', 2),
(5, 'Ethan', 'Thomas', 2),
(8, 'Fiona', 'Roberts', 3);
```

Creating FOREIGN KEY constraints

```
-- Creating a Foreign Key Constraint
ALTER TABLE School.ScoresCopy WITH NOCHECK
ADD CONSTRAINT FK_ScoresCopy_PupilsCopy
FOREIGN KEY (PupilID)
REFERENCES School.PupilsCopy(PupilID)
ON UPDATE NO ACTION
ON DELETE NO ACTION

ALTER TABLE School.ScoresCopy
DROP CONSTRAINT FK_ScoresCopy_PupilsCopy

ALTER TABLE School.ScoresCopy
NOCHECK CONSTRAINT FK_ScoresCopy_PupilsCopy

ALTER TABLE School.ScoresCopy
WITH CHECK CHECK CONSTRAINT FK_ScoresCopy_PupilsCopy

ALTER TABLE School.ScoresCopy
WITH NOCHECK CHECK CONSTRAINT FK_ScoresCopy_PupilsCopy

ALTER TABLE School.ScoresCopy
DROP CONSTRAINT FK_ScoresCopy_PupilsCopy


-- Using CASCADE
ALTER TABLE School.ScoresCopy WITH NOCHECK
ADD CONSTRAINT FK_ScoresCopy_PupilsCopy
FOREIGN KEY (PupilID)
```



```
REFERENCES School.PupilsCopy(PupilID)
ON UPDATE CASCADE
ON DELETE CASCADE

ALTER TABLE School.ScoresCopy
DROP CONSTRAINT FK_ScoresCopy_PupilsCopy

-- Using SET NULL
ALTER TABLE School.ScoresCopy WITH NOCHECK
ADD CONSTRAINT FK_ScoresCopy_PupilsCopy
FOREIGN KEY (PupilID)
REFERENCES School.PupilsCopy(PupilID)
ON UPDATE SET NULL
ON DELETE SET NULL

ALTER TABLE School.ScoresCopy
DROP CONSTRAINT FK_ScoresCopy_PupilsCopy

-- Using SET DEFAULT
INSERT INTO School.PupilsCopy (PupilID, FirstName, LastName, ClassID)
VALUES (999, 'Unknown', 'Unknown', 0)

ALTER TABLE School.ScoresCopy
ADD CONSTRAINT DK_ScoresCopy_PupilID DEFAULT 999 FOR PupilID

ALTER TABLE School.ScoresCopy WITH NOCHECK
ADD CONSTRAINT FK_ScoresCopy_PupilsCopy
FOREIGN KEY (PupilID)
REFERENCES School.PupilsCopy(PupilID)
ON UPDATE SET DEFAULT
ON DELETE SET DEFAULT
```

Updating Data

```
SELECT * FROM School.ScoresCopy
SELECT * FROM School.PupilsCopy

UPDATE School.ScoresCopy
SET PupilID = 8
WHERE PupilID = 7

UPDATE School.ScoresCopy
SET PupilID = 7
WHERE PupilID = 8

UPDATE School.PupilsCopy
SET PupilID = 7
WHERE PupilID = 8

UPDATE School.PupilsCopy
SET PupilID = 8
WHERE PupilID = 7
```



```
DELETE FROM School.PupilsCopy
WHERE PupilID = 2
```

Implement programmability objects and Write advanced T-SQL code

7. Create views

Creating a simple view

```
SELECT * FROM [School].[Scores]

DROP VIEW IF EXISTS [School].[vScoresMath]
GO

CREATE OR ALTER VIEW [School].[vScoresMath] AS
SELECT *
FROM [School].[Scores]
WHERE SetType = 'Math'
GO

SELECT * FROM [School].[vScoresMath]
ORDER BY PupilID

INSERT INTO [School].[vScoresMath](PupilID, Score, SetType, DateTaken,
Maximum)
VALUES (90, 90, 'Math', '2024-06-01', 100)

DELETE FROM [School].[vScoresMath]
WHERE PupilID = 90

-- System views
SELECT * FROM sys.views
SELECT * FROM sys.columns WHERE object_id = OBJECT_ID('School.vScoresMath')
SELECT * FROM sys.syscomments WHERE id = OBJECT_ID('School.vScoresMath')
SELECT * FROM sys.sql_expression_dependencies AS sed
WHERE referencing_id = OBJECT_ID('School.vScoresMath')
```

Expanding the view

```
DROP VIEW IF EXISTS [School].[vScoresMath]
GO

CREATE OR ALTER VIEW [School].[vScoresMath] AS
SELECT *
FROM [School].[Scores]
WHERE SetType = 'Math'
GO

SELECT * FROM [School].[vScoresMath]
ORDER BY PupilID
```



```
SELECT * FROM sys.syscomments WHERE id = OBJECT_ID('School.vScoresMath')

-- With Encryption
CREATE OR ALTER VIEW [School].[vScoresMath](PupilID, Score, SetType,
DateTaken, Maximum, DateRecorded, ScoresID)
WITH ENCRYPTION
AS
SELECT * FROM [School].[Scores]
WHERE SetType = 'Math'

SELECT * FROM sys.syscomments WHERE id = OBJECT_ID('School.vScoresMath')

-- With Schema Binding
CREATE OR ALTER VIEW [School].[vScoresMath](PupilID, Score, SetType,
DateTaken, Maximum, DateRecorded, ScoresID)
WITH SCHEMABINDING
AS
SELECT PupilID, Score, SetType, DateTaken, Maximum, DateRecorded, ScoresID
FROM [School].[Scores]
WHERE SetType = 'Math'

DROP TABLE School.Scores -- This will fail because the view is schemabound

-- With View_Metadata
CREATE OR ALTER VIEW [School].[vScoresMath](PupilID, Score, SetType,
DateTaken, Maximum, DateRecorded, ScoresID)
WITH VIEW_METADATA
AS
SELECT PupilID, Score, SetType, DateTaken, Maximum, DateRecorded, ScoresID
FROM [School].[Scores]
WHERE SetType = 'Math'

-- With Check Option
CREATE OR ALTER VIEW [School].[vScoresMath](PupilID, Score, SetType,
DateTaken, Maximum, DateRecorded, ScoresID)
AS
SELECT PupilID, Score, SetType, DateTaken, Maximum, DateRecorded, ScoresID
FROM [School].[Scores]
WHERE SetType = 'Math'
WITH CHECK OPTION -- Additional option

SELECT * FROM [School].[vScoresMath]

INSERT INTO [School].[vScoresMath](PupilID, Score, SetType, DateTaken,
Maximum, DateRecorded)
VALUES (90, 90, 'Math', '2024-06-01', 100, GETDATE())

DELETE FROM [School].[vScoresMath]
WHERE PupilID = 90

INSERT INTO [School].[vScoresMath](PupilID, Score, SetType, DateTaken,
Maximum, DateRecorded)
```



```
VALUES (90, 90, 'English', '2024-06-01', 100, GETDATE()) -- Does not work
because of the check option
```

Multiple tables

```
DELETE FROM [School].[Scores]
WHERE PupilID = 90
GO

CREATE OR ALTER VIEW [School].[vScoresMath]
AS
SELECT COALESCE(P.PupilID, S.PupilID) AS PupilID,
       -- P.PupilID AS PupilIDFromPupils, S.PupilID as PupilIDFromScores,
       Score, SetType, DateTaken, Maximum, DateRecorded, ScoresID,
       FirstName, LastName, ClassID
FROM [School].[Scores] AS S
FULL JOIN School.Pupils AS P
ON S.PupilID = P.PupilID
WHERE SetType = 'Math'
GO

SELECT * FROM [School].[vScoresMath]
ORDER BY PupilID

INSERT INTO [School].[vScoresMath](PupilIDFromScores, Score, SetType,
DateTaken, Maximum
            , PupilIDFromPupils, FirstName, LastName, ClassID)
VALUES (90, 90, 'Math', '2024-06-01', 100, 90, 'John', 'Doe', 999)

INSERT INTO [School].[vScoresMath](PupilID, Score, SetType, DateTaken,
Maximum)
VALUES (90, 90, 'Math', '2024-06-01', 100, 90)
```

11. Create triggers

```
SELECT * FROM School.Pupils;
SELECT * FROM School.Scores;

DROP TABLE [School].[PupilsCopy]
SELECT *
INTO [School].[PupilsCopy]
FROM School.Pupils;

SELECT * FROM School.PupilsCopy;

DELETE FROM [School].[Scores] WHERE PupilID = 90
DELETE FROM [School].[Pupils] WHERE PupilID = 90
DELETE FROM [School].[PupilsCopy] WHERE PupilID = 90
DROP TRIGGER IF EXISTS School.trgInsertScoresMath
DROP TRIGGER IF EXISTS School.trgUpdateScoresMath
GO

CREATE OR ALTER VIEW [School].[vScoresMath]
```



```
AS
SELECT COALESCE(P.PupilID, S.PupilID) AS PupilID,
       Score, SetType, DateTaken, Maximum, DateRecorded, ScoresID,
       FirstName, LastName, ClassID
FROM [School].[Scores] AS S
FULL JOIN School.Pupils AS P
ON S.PupilID = P.PupilID
WHERE SetType = 'Math'
GO

INSERT INTO [School].[vScoresMath](PupilID, Score, SetType, DateTaken,
Maximum, FirstName, LastName, ClassID)
VALUES (90, 90, 'Math', '2034-06-01', 100, 'John', 'Doe', 999)

UPDATE [School].[vScoresMath]
SET Score = 95, FirstName = 'Jane', LastName = 'Smith', ClassID = 998
WHERE PupilID = 90
GO

SELECT * FROM [School].[vScoresMath]
GO

CREATE OR ALTER TRIGGER School.trgInsertScoresMath
ON [School].[vScoresMath]
INSTEAD OF INSERT
AS
BEGIN
    INSERT INTO School.Pupils (PupilID, FirstName, LastName, ClassID)
    SELECT PupilID, FirstName, LastName, ClassID
    FROM inserted
    WHERE PupilID NOT IN (SELECT PupilID FROM School.Pupils);

    INSERT INTO School.Scores (PupilID, Score, SetType, DateTaken, Maximum,
DateRecorded)
    SELECT PupilID, Score, SetType, DateTaken, Maximum, GETDATE()
    FROM inserted
    WHERE PupilID IS NOT NULL;
END

CREATE OR ALTER TRIGGER School.trgUpdateScoresMath
on [School].[vScoresMath]
INSTEAD OF UPDATE
AS
BEGIN
    UPDATE P
    SET P.FirstName = inserted.FirstName,
        P.LastName = inserted.LastName,
        P.ClassID = inserted.ClassID
    FROM School.Pupils AS P
    INNER JOIN inserted ON P.PupilID = inserted.PupilID;

    UPDATE S
    SET S.Score = inserted.Score,
```



```
        S.SetType = inserted.SetType,
        S.DateTaken = inserted.DateTaken,
        S.Maximum = inserted.Maximum,
        S.DateRecorded = GETDATE()
    FROM School.Scores AS S
    INNER JOIN inserted ON S.PupilID = inserted.PupilID;

    INSERT INTO School.PupilsCopy (PupilID, FirstName, LastName, ClassID)
    SELECT PupilID, FirstName, LastName, ClassID
    FROM deleted;
END
```

18. Write correlated queries

Derived tables

```
CREATE OR ALTER VIEW [School].[vScoresMath] AS
SELECT * FROM [School].[Scores]
WHERE SetType = 'Math'
GO

SELECT * FROM [School].Pupils
SELECT * FROM [School].[vScoresMath]

-- the FROM clause
SELECT *
FROM School.Pupils AS P
LEFT JOIN School.vScoresMath AS S
ON P.PupilID = S.PupilID

SELECT *
FROM School.Pupils AS P
LEFT JOIN (SELECT * FROM [School].[Scores]
WHERE SetType = 'Math') AS S
ON P.PupilID = S.PupilID

SELECT *
FROM School.Pupils AS P
WHERE PupilID IN (SELECT PupilID FROM [School].[Scores] WHERE SetType =
'Math')
```

Subqueries in the SELECT and WHERE clauses

```
SELECT AVG(Score) AS AverageScore
FROM [School].[Scores] AS S
WHERE SetType = 'Math'

SELECT *, (SELECT AVG(Score) AS AverageScore
FROM [School].[Scores] AS S
WHERE SetType = 'Math') AS AverageScore
FROM [School].[vScoresMath] AS S
WHERE Score > (SELECT AVG(Score) AS AverageScore
FROM [School].[Scores] AS S
WHERE SetType = 'Math')
```


Correlated queries

```
-- Correlated subquery in the SELECT clause
CREATE OR ALTER VIEW [School].[vScoresMath] AS
SELECT * FROM [School].[Scores]
WHERE SetType = 'Math'
GO

SELECT * FROM [School].[vScoresMath]

SELECT P.*, AVG(Score) AS AverageScore
FROM [School].Pupils AS P
LEFT JOIN [School].[vScoresMath] AS S
ON P.PupilID = S.PupilID
GROUP BY P.PupilID, P.FirstName, P.LastName, P.ClassID

SELECT *, (SELECT AVG(Score) FROM School.vScoresMath AS S WHERE P.PupilID =
S.PupilID) AS AverageScore
FROM [School].Pupils AS P



-- Correlated column in the WHERE clause

SELECT P.*
FROM [School].Pupils AS P
LEFT JOIN [School].[vScoresMath] AS S
ON P.PupilID = S.PupilID
WHERE S.Score > 80

SELECT *
FROM School.Pupils as P
WHERE EXISTS (SELECT 1 FROM School.vScoresMath AS S WHERE P.PupilID =
S.PupilID AND S.Score > 80)

SELECT *
FROM School.Pupils as P
WHERE PupilID IN (SELECT PupilID FROM School.vScoresMath AS S WHERE
P.PupilID = S.PupilID AND S.Score > 80)
```

13. Write queries that include window functions

ROW_NUMBER, RANK, DENSE_RANK and NTILE

```
SELECT Score, SetType,
       ROW_NUMBER() OVER (PARTITION BY SetType ORDER BY Score) AS RowNum,
       RANK()       OVER (PARTITION BY SetType ORDER BY Score) AS RankNum,
       DENSE_RANK() OVER (PARTITION BY SetType ORDER BY Score) AS
DenseRankNum,
       NTILE(4)     OVER (PARTITION BY SetType ORDER BY Score) AS Quartile
FROM School.Scores
ORDER BY SetType, Score;
```



```
SELECT Score, SetType,
       ROW_NUMBER() OVER w AS RowNum,
       RANK()       OVER w AS RankNum,
       DENSE_RANK() OVER w AS DenseRankNum,
       NTILE(4)     OVER w AS Quartile
FROM School.Scores
WINDOW w AS (PARTITION BY SetType ORDER BY Score)
ORDER BY SetType, Score;
```

LAG and LEAD

```
SELECT Score, SetType, LAG(Score) OVER (ORDER BY Score) AS PreviousScore
                     , LEAD(Score) OVER (ORDER BY Score) AS NextScore
                     , Score - LAG(Score) OVER (ORDER BY Score) AS
DifferenceFromPrevious
FROM School.Scores
ORDER BY Score;

SELECT Score, SetType, LAG(Score, 2, 0) OVER (ORDER BY Score) AS
PreviousScore
                     , LEAD(Score, 2, 999) OVER (ORDER BY Score) AS
NextScore
FROM School.Scores
ORDER BY Score;

SELECT Score, SetType, LAG(Score) IGNORE NULLS OVER (PARTITION BY SetType
ORDER BY Score) AS PreviousScore
                     , LEAD(Score) RESPECT NULLS OVER (PARTITION BY SetType
ORDER BY Score) AS NextScore
FROM School.Scores
ORDER BY SetType, Score;
```

FIRST_VALUE and LAST_VALUE

```
SELECT Score, SetType, FIRST_VALUE(Score) OVER w AS FirstScore
                              , LAST_VALUE(Score)  OVER W AS LastScore
FROM School.Scores
WINDOW w AS (ORDER BY Score RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
FOLLOWING)
ORDER BY Score;

SELECT Score, SetType, FIRST_VALUE(SetType) OVER w AS FirstScore
                              , LAST_VALUE(SetType)  OVER W AS LastScore
FROM School.Scores
WINDOW w AS (ORDER BY Score ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)
ORDER BY Score;

SELECT Score, SetType, FIRST_VALUE(Score) OVER w AS FirstScore
                              , LAST_VALUE(Score)  OVER W AS LastScore
FROM School.Scores
WINDOW w AS (ORDER BY Score RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW)
ORDER BY Score;
```



```
SELECT Score, SetType, ScoresID
, LAST_VALUE(ScoresID) OVER (ORDER BY Score RANGE BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW) AS LastRange
, LAST_VALUE(ScoresID) OVER (ORDER BY Score ROWS  BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW) AS LastRows
FROM School.Scores
ORDER BY Score;
```

CUME_DIST, PERCENT_RANK, PERCENTILE_CONT and PERCENTILE_DISC

```
SELECT SetType, Score, ROW_NUMBER() OVER w AS RowNum
                          , CUME_DIST() OVER w AS CumeDist
                          , PERCENT_RANK() OVER w AS PercentRank
FROM School.Scores
WINDOW w AS (PARTITION BY SetType ORDER BY Score)
ORDER BY SetType, Score

SELECT SetType, AVG(Score * 1.0) AS AverageScore
FROM School.Scores
GROUP BY SetType
ORDER BY Score

SELECT SetType, Score
FROM School.Scores
ORDER BY SetType, Score

SELECT DISTINCT SetType,
                PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY Score) OVER
(PARTITION BY SetType) AS MedianScore,
                        PERCENTILE_DISC(0.5) WITHIN GROUP (ORDER BY Score)
OVER (PARTITION BY SetType) AS MedianDisc
FROM School.Scores

SELECT SetType, AVG(Score * 1.0) AS AverageScore
FROM School.Scores
ORDER BY Score
```

Aggregations

```
SELECT SetType, Score
     , (SELECT AVG(Score * 1.0) FROM School.Scores AS S2 WHERE S.SetType =
S2.SetType) AS AverageScore
     , AVG(Score * 1.0) OVER (PARTITION BY SetType ORDER BY Score
                              ROWS BETWEEN UNBOUNDED PRECEDING AND
UNBOUNDED FOLLOWING) AS AverageScoreWindow
     , SUM(Score) OVER (PARTITION BY SetType ORDER BY Score
                ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
TotalScoreWindow1
     , SUM(Score) OVER (PARTITION BY SetType ORDER BY Score
                RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
TotalScoreWindow2
     , COUNT(Score) OVER (PARTITION BY SetType ORDER BY Score
                ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
TotalScoreWindow
```



```
FROM School.Scores AS S
ORDER BY SetType, Score

SELECT SetType, AVG(Score * 1.0) AS AverageScore
FROM School.Scores AS S
GROUP BY SetType

SELECT SetType, COUNT(*) AS TotalScores, COUNT(Score) AS TotalNonNullScores
              , COUNT(DISTINCT Score) AS TotalDistinctScores
FROM School.Scores AS S
GROUP BY SetType

SELECT SetType, Score
     , (SELECT AVG(Score * 1.0) FROM School.Scores AS S2 WHERE S.SetType =
S2.SetType) AS AverageScore
     , AVG(Score * 1.0) OVER (PARTITION BY SetType ORDER BY Score
                              ROWS BETWEEN UNBOUNDED PRECEDING AND
UNBOUNDED FOLLOWING) AS AverageScoreWindow
     , SUM(Score) OVER (PARTITION BY SetType ORDER BY Score
                  ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) AS
SumScoreWindow
     , COUNT(Score) OVER (PARTITION BY SetType ORDER BY Score
                  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
TotalScoreWindow
FROM School.Scores AS S
ORDER BY SetType, Score
```

12. Write common table expressions (CTEs)

Non-recursive CTE

```
WITH CTE_Scores AS
(SELECT Score, SetType, ROW_NUMBER() OVER (ORDER BY Score) AS RowNum
 FROM School.Scores)
SELECT *
FROM CTE_Scores
WHERE RowNum BETWEEN 3 AND 7
ORDER BY RowNum;

WITH CTE_Scores AS
(SELECT Score, SetType, ROW_NUMBER() OVER (ORDER BY Score) AS RowNum
 FROM School.Scores),
CTE_ScoresBetween3And7(Score, SetType, RowNumber) AS
(SELECT *
 FROM CTE_Scores
 WHERE RowNum BETWEEN 3 AND 7)
SELECT *
FROM CTE_ScoresBetween3And7
ORDER BY RowNumber

SELECT Score, SetType, ROW_NUMBER() OVER (ORDER BY Score) AS RowNum
FROM School.Scores
```



```
ORDER BY RowNum
OFFSET 2 ROWS FETCH NEXT 5 ROWS ONLY
```

UNION, UNION ALL, EXCEPT and INTERSECT operators

```
SELECT PupilID AS PupilNum, FirstName, LastName, ClassID
FROM School.Pupils
WHERE PupilID IN (3, 4, 5)
EXCEPT
SELECT PupilID, FirstName, LastName, ClassID
FROM School.Pupils
WHERE PupilID IN (1, 2, 3);

SELECT PupilID, FirstName, LastName, ClassID
FROM School.Pupils
WHERE PupilID IN (1, 2, 3)
EXCEPT
SELECT PupilID AS PupilNum, FirstName, LastName, ClassID * 2
FROM School.Pupils
WHERE PupilID IN (3, 4, 5)

(SELECT PupilID AS PupilNum, FirstName, LastName, ClassID
FROM School.Pupils
WHERE PupilID IN (3, 4, 5)
union
SELECT PupilID, FirstName, LastName, ClassID
FROM School.Pupils
WHERE PupilID IN (1, 2, 3))
UNION ALL
SELECT PupilID, FirstName, LastName, ClassID
FROM School.Pupils
WHERE PupilID IN (5, 6, 7, 8)
```

Recursive CTE

```
SELECT *
FROM [School].[Pupils]

DROP TABLE IF EXISTS School.PupilFriends
GO

CREATE TABLE School.PupilFriends
(PupilID int,
      FriendID int,
      PRIMARY KEY (PupilID, FriendID),
      FOREIGN KEY (PupilID) REFERENCES School.Pupils(PupilID),
      FOREIGN KEY (FriendID) REFERENCES School.Pupils(PupilID)
)

INSERT INTO School.PupilFriends
VALUES (1, 2),
(1, 3),
(2, 5),
```



```
(3, 4),
(4, 5),
(5, 8)

SELECT * FROM School.PupilFriends;
-- Longhand CTE to find all friends of friends for PupilID = 1
WITH
L0 AS
(
    -- Level 0: start pupil (the anchor row)
    SELECT pf.PupilID, pf.PupilID AS FriendID, 0 AS LevelRemoved
    FROM School.PupilFriends pf
    WHERE pf.PupilID = 1),
L1 AS
(
    -- Level 1: direct friends of start pupil
    SELECT pf.PupilID, pf.FriendID, 0+1 AS LevelRemoved
    FROM School.PupilFriends pf
    WHERE pf.PupilID = 1
),
L2 AS
(
    -- Level 2: friends of Level 1
    SELECT pf.PupilID, pf.FriendID, 1+1 AS LevelRemoved
    FROM School.PupilFriends pf
    JOIN L1
    ON pf.PupilID = L1.FriendID
),
L3 AS
(
    -- Level 3: friends of Level 2
    SELECT pf.PupilID, pf.FriendID, 2+1 AS LevelRemoved
    FROM School.PupilFriends pf
    JOIN L2
    ON pf.PupilID = L2.FriendID
),
L4 AS
(
    -- Level 4: friends of Level 3
    SELECT pf.PupilID, pf.FriendID, 3+1 AS LevelRemoved
    FROM School.PupilFriends pf
    JOIN L3
    ON pf.PupilID = L3.FriendID
),
AllLevels AS
(
    SELECT * FROM L0
    UNION ALL SELECT * FROM L1
    UNION ALL SELECT * FROM L2
    UNION ALL SELECT * FROM L3
    UNION ALL SELECT * FROM L4
)
```



```
SELECT FriendID, MIN(LevelRemoved) AS LevelsRemoved, (SELECT FirstName FROM
School.Pupils WHERE PupilID = FriendID) AS FriendFirstName
FROM AllLevels
GROUP BY FriendID
ORDER BY LevelsRemoved, FriendID;

-- CTE to find all friends of friends for PupilID = 1
WITH RecursiveFriends AS
(
    -- Anchor (level 0 = the starting pupil)
    SELECT PupilID, PupilID AS FriendID, 0 AS LevelRemoved
    FROM School.PupilFriends AS P
    WHERE PupilID = 1

    UNION ALL

    -- Recursive (add 1 level each step)
    SELECT P.PupilID, P.FriendID, RF.LevelRemoved + 1
    FROM School.PupilFriends P
    JOIN RecursiveFriends RF
    ON P.PupilID = RF.FriendID
)
SELECT FriendID, MIN(LevelRemoved) AS LevelsRemoved, (SELECT FirstName FROM
School.Pupils WHERE PupilID = FriendID) AS FriendFirstName
FROM RecursiveFriends
GROUP BY FriendID
ORDER BY LevelsRemoved, FriendID
```

10. Create stored procedures

```
CREATE PROC School.PupilsAndScores
AS
BEGIN
SELECT * FROM School.Pupils
SELECT * FROM School.Scores
END

EXEC School.PupilsAndScores
GO

CREATE OR ALTER PROC School.PupilsAndScores @PupilID INT = 1
AS
BEGIN
SELECT * FROM School.Pupils WHERE PupilID = @PupilID
SELECT * FROM School.Scores WHERE PupilID = @PupilID
END

EXEC School.PupilsAndScores
EXEC School.PupilsAndScores 2
EXEC School.PupilsAndScores @PupilID = 3
GO

CREATE OR ALTER PROC School.PupilsAndScores
```



```
@PupilID INT = 1, @FirstName VARCHAR(50) OUTPUT, @LastName VARCHAR(50)
OUTPUT,
@AverageScore DECIMAL(4,1) OUTPUT
AS
BEGIN
SELECT * FROM School.Pupils WHERE PupilID = @PupilID
SELECT * FROM School.Scores WHERE PupilID = @PupilID

SELECT @FirstName = FirstName, @LastName = LastName
FROM School.Pupils WHERE PupilID = @PupilID
SELECT @AverageScore = AVG(Score*1.0)
FROM School.Scores WHERE PupilID = @PupilID
END

GO

DECLARE @FirstName VARCHAR(50)
DECLARE @LastName VARCHAR(50)
DECLARE @AverageScore DECIMAL(4,1)
EXEC School.PupilsAndScores 1, @FirstName OUTPUT, @LastName OUTPUT,
@AverageScore OUTPUT
SELECT @FirstName AS FirstName, @LastName AS LastName, @AverageScore AS
AverageScore
GO
```

19. Implement error handling

```
CREATE OR ALTER PROC School.PupilsAndScores
@PupilID INT = 1, @FirstName VARCHAR(50) OUTPUT, @LastName VARCHAR(50)
OUTPUT,
@AverageScore DECIMAL(4,1) OUTPUT
AS
BEGIN
SELECT * FROM School.Pupils WHERE PupilID = @PupilID
SELECT * FROM School.Scores WHERE PupilID = @PupilID

SELECT @FirstName = FirstName, @LastName = LastName
FROM School.Pupils WHERE PupilID = @PupilID
SELECT @AverageScore = ISNULL(SUM(Score*1.0), 0) / COUNT(*)
FROM School.Scores WHERE PupilID = @PupilID
END

GO


DECLARE @FirstName VARCHAR(50)
DECLARE @LastName VARCHAR(50)
DECLARE @AverageScore DECIMAL(4,1)
EXEC School.PupilsAndScores  9, @FirstName OUTPUT, @LastName OUTPUT,
@AverageScore OUTPUT
SELECT @FirstName AS FirstName, @LastName AS LastName, @AverageScore AS
AverageScore
GO
```



```
-- Add TRY and CATCH block to handle potential errors
CREATE OR ALTER PROC School.PupilsAndScores
@PupilID INT = 1, @FirstName VARCHAR(200) OUTPUT, @LastName VARCHAR(50)
OUTPUT,
@AverageScore DECIMAL(4,1) OUTPUT
AS
BEGIN
SELECT * FROM School.Pupils WHERE PupilID = @PupilID
SELECT * FROM School.Scores WHERE PupilID = @PupilID

SELECT @FirstName = FirstName, @LastName = LastName
FROM School.Pupils WHERE PupilID = @PupilID

BEGIN TRY
      SELECT @AverageScore = ISNULL(SUM(Score*1.0), 0) / COUNT(*)
      FROM School.Scores WHERE PupilID = @PupilID
      RETURN 0
END TRY
BEGIN CATCH
    SET @FirstName = 'Number: ' + CAST(ERROR_NUMBER() AS NVARCHAR(100))
                     + ', Message: ' + CAST(ERROR_MESSAGE() AS
NVARCHAR(100))
                           + ', Procedure: ' + CAST(ERROR_PROCEDURE() AS
NVARCHAR(100))
      Set @LastName = 'Line: ' + CAST(ERROR_LINE() AS NVARCHAR(100))
                           + ', Severity: ' + CAST(ERROR_SEVERITY() AS
NVARCHAR(100))
                           + ', State: ' + CAST(ERROR_STATE() AS
NVARCHAR(100))
      SET @AverageScore = 0
      RETURN 1
END CATCH
END
GO

DECLARE @FirstName VARCHAR(200)
DECLARE @LastName VARCHAR(50)
DECLARE @AverageScore DECIMAL(4,1)
DECLARE @Result INT
EXEC @Result = School.PupilsAndScores 1, @FirstName OUTPUT, @LastName
OUTPUT, @AverageScore OUTPUT
SELECT @Result AS Result, @FirstName AS FirstName, @LastName AS LastName,
@AverageScore AS AverageScore
GO

---

CREATE OR ALTER PROC School.PupilsAndScores
@PupilID INT = 1, @FirstName VARCHAR(200) OUTPUT, @LastName VARCHAR(50)
OUTPUT,
@AverageScore DECIMAL(4,1) OUTPUT
AS
```



```
BEGIN
SELECT * FROM School.Pupils WHERE PupilID = @PupilID
SELECT * FROM School.Scores WHERE PupilID = @PupilID

SELECT @FirstName = FirstName, @LastName = LastName
FROM School.Pupils WHERE PupilID = @PupilID

      DECLARE @NumberOfRecords INT
      SELECT @NumberOfRecords = COUNT(*) FROM School.Scores WHERE PupilID =
@PupilIDInt

      IF @NumberOfRecords <> 0
      BEGIN
            SELECT @AverageScore = ISNULL(SUM(Score*1.0), 0)/COUNT(*) FROM
School.Scores WHERE PupilID = @PupilIDInt
            SET NOCOUNT OFF;
            RETURN 0
      END
      ELSE
      BEGIN
            RETURN 1
      END
END
GO
```

8. Create scalar functions

```
CREATE OR ALTER FUNCTION School.fnGetPupilName(@PupilID INT)
RETURNS VARCHAR(100)
AS
BEGIN
      DECLARE @Name VARCHAR(100);

      SELECT @Name = FirstName + ' ' + LastName
      FROM School.Pupils
      WHERE PupilID = @PupilID;

      RETURN @Name;
END
GO

SELECT School.fnGetPupilName(1) AS PupilName;

SELECT *, School.fnGetPupilName(PupilID) AS FullName
FROM School.Pupils;

ALTER TABLE School.Pupils
ADD FullName AS School.fnGetPupilName(PupilID);

SELECT * FROM School.Pupils;

ALTER TABLE School.Pupils
DROP COLUMN FullName;
```



```
SELECT * FROM sys.sql_modules
WHERE object_id = OBJECT_ID('School.fnGetPupilName');

DROP FUNCTION School.fnGetPupilName;
```

9. Create table-valued functions

```
CREATE OR ALTER FUNCTION School.fnGetPupilScores(@PupilID INT)
RETURNS TABLE
AS
RETURN
(
      SELECT SetType, AVG(Score) AS AverageScore, COUNT(*) AS
NumberOfScores
      FROM School.Scores
      WHERE PupilID = @PupilID
      GROUP BY SetType
)
GO

SELECT * FROM School.fnGetPupilScores(2)

SELECT * FROM School.Pupils AS P
OUTER APPLY School.fnGetPupilScores(P.PupilID) AS S;

DROP FUNCTION School.fnGetPupilScores;
GO

CREATE OR ALTER FUNCTION School.fnGetPupilScores(@PupilID INT)
RETURNS @tblScores TABLE
(SetType VARCHAR(50),
AverageScore DECIMAL(5, 2),
NumberOfScores INT)
AS
BEGIN
    INSERT INTO @tblScores (SetType, AverageScore, NumberOfScores)
      SELECT SetType, AVG(Score) AS AverageScore, COUNT(*) AS
NumberOfScores
      FROM School.Scores
      WHERE PupilID = @PupilID
      GROUP BY SetType
      RETURN
END
GO
```


JSON

14. Write queries that include JSON functions, such as

Introducing JSON

```
DECLARE @jsonDocument JSON = '{"Given name":"Phillip","Family
name":"Burton"}'
SELECT @jsonDocument

DECLARE @jsonDocument JSON = '{a"Given name":"Phillip","Family
name":"Burton"}'
SELECT @jsonDocument

DECLARE @jsonDocument JSON = '[{"Given name":"Phillip","Family
name":"Burton"}, {"Given name":"Jane","Family name":"Smith"}]'
SELECT @jsonDocument

DECLARE @jsonDocument JSON = '{"Names": [{"Given name":"Phillip","Family
name":"Burton"}, {"Given name":"Jane","Family name":"Smith"}]}'
SELECT @jsonDocument
```

JSON_OBJECT

```
SELECT JSON_OBJECT('Given name': 'Phillip', 'Family name': 'Burton')
SELECT JSON_OBJECT('Given name': 'Phillip', 'Family name': 'Burton'
RETURNING json)
SELECT JSON_OBJECT('Given name': 'Phillip', 'Middle name': NULL, 'Family
name': 'Burton' NULL ON NULL)
SELECT JSON_OBJECT('Given name': 'Phillip', 'Middle name': NULL, 'Family
name': 'Burton' ABSENT ON NULL)

DECLARE @jsondocument json = JSON_OBJECT('Given name':'Phillip', 'Family
name': 'Burton')
SELECT @jsondocument;
```

JSON_ARRAY

```
DECLARE @jsondocument json = JSON_ARRAY(
  JSON_OBJECT('Given name':'Phillip', 'Family name':'Burton')
, JSON_OBJECT('Given name':'Jane', 'Family name':'Smith'))
SELECT @jsondocument;
```

JSON_ARRAYAGG

```
SELECT PupilID, SetType
FROM School.Scores as S;

SELECT PupilID, JSON_ARRAYAGG(S.SetType) AS SetTypes
FROM School.Scores as S
GROUP BY PupilID;

SELECT PupilID, JSON_ARRAYAGG(S.SetType ORDER BY SetType) AS SetTypes
FROM School.Scores as S
```



```
GROUP BY PupilID;
```

JSON_CONTAINS

```
DECLARE @jsondocument json = JSON_ARRAY(
  JSON_OBJECT('Given name':'Phillip', 'Family name':'Burton')
, JSON_OBJECT('Given name':'Jane', 'Family name':'Smith'))
SELECT @jsondocument;
SELECT JSON_CONTAINS(@jsondocument, 'Phillip', '$.*')
SELECT JSON_CONTAINS(@jsondocument, 'Phil', '$.*')
SELECT JSON_CONTAINS(@jsondocument, 'Phil%', '$."Given name"', 0)
SELECT JSON_CONTAINS(@jsondocument, 'Phil%', '$."Given name"', 1)

DECLARE @jsondocument2 json = JSON_OBJECT('Name': JSON_OBJECT('Given
name':'Phillip', 'Family name': 'Burton'))
SELECT @jsondocument2;
SELECT JSON_CONTAINS(@jsondocument2, 'Phillip', '$.Name."Given name"')


DECLARE @jsondocument3 json = JSON_ARRAY(
  JSON_OBJECT('Given name':'Phillip', 'Family name':'Burton')
, JSON_OBJECT('Given name':'Jane', 'Family name':'Smith'))
SELECT @jsondocument3;

SELECT
    JSON_CONTAINS(@jsondocument3, 'Trevor', '$[*]."Given name"') AS
ContainsGivenName;
```

OPENJSON

```
DECLARE @jsondocument json = JSON_OBJECT('Given name':'Phillip', 'Family
name': 'Burton')
SELECT * FROM OPENJSON(@jsondocument)

DECLARE @jsondocument json = JSON_ARRAY(
  JSON_OBJECT('Given name':'Phillip', 'Family name':'Burton')
, JSON_OBJECT('Given name':'Jane', 'Family name':'Smith'))

SELECT * FROM OPENJSON(@jsondocument)

SELECT * FROM OPENJSON(@jsondocument)
WITH (
    [Given name]  NVARCHAR(100) '$."Given name"',
    [Family name] NVARCHAR(100) '$."Family name"'
)
```

JSON_VALUE

```
DECLARE @jsondocument json = JSON_OBJECT('Given name':'Phillip', 'Family
name':'Burton', 'Height':160.5, 'Date of birth':'1983-01-02T03:04:05');

SELECT JSON_VALUE(@jsondocument, '$."Given name"')
SELECT JSON_VALUE(@jsondocument, '$."Height"' RETURNING INT)
SELECT JSON_VALUE(@jsondocument, '$."Date of birth"' RETURNING DATE)
GO
```



```
DECLARE @jsondocument json = JSON_ARRAY(
  JSON_OBJECT('Given name':'Phillip', 'Family name':'Burton'),
  JSON_OBJECT('Given name':'Jane', 'Family name':'Smith')
);

SELECT JSON_VALUE(@jsondocument, '$[0]."Given name"')
```

3. Design and implement JSON columns and indexes

Creating JSON Columns

```
DROP TABLE IF EXISTS [School].[PupilsCopy];

CREATE TABLE [School].[PupilsCopy](
      [PupilID] [int] NOT NULL PRIMARY KEY,
      [FirstName] [varchar](50) NULL,
      [LastName] [varchar](50) NULL,
      [ClassID] [int] NULL)
GO

INSERT INTO [School].[PupilsCopy](PupilID, FirstName, LastName, ClassID)
SELECT PupilID, FirstName, LastName, ClassID
FROM [School].[Pupils];
GO

ALTER TABLE [School].[PupilsCopy]
ADD JSONNames JSON;

UPDATE [School].[PupilsCopy]
SET JSONNames = JSON_OBJECT('Given Name':FirstName, 'Family
Name':LastName);

SELECT * FROM [School].[PupilsCopy]
WHERE JSON_CONTAINS(JSONNames, 'Fiona', '$.*') = 1;

SELECT * FROM [School].[PupilsCopy]
WHERE JSON_VALUE(JSONNames, '$."Given Name"') = 'Fiona';
```

Creating JSON Index

```
CREATE JSON INDEX idx_JSONNames ON [School].[PupilsCopy](JSONNames); -- SQL
Server 2025 onwards

SELECT JSON_VALUE(JSONNames, '$."Given Name"')
FROM [School].[PupilsCopy]

ALTER TABLE [School].[PupilsCopy]
ADD GivenNameFromJSON AS JSON_VALUE(JSONNames, '$."Given Name"') PERSISTED;

CREATE INDEX idx_GivenNameFromJSON ON
[School].[PupilsCopy](GivenNameFromJSON);
```



```
ALTER TABLE [School].[PupilsCopy]
DROP COLUMN GivenNameFromJSON;

ALTER TABLE [School].[PupilsCopy]
ADD GivenNameFromJSON AS CAST(JSON_VALUE(JSONNames, '$."Given Name"') AS
NVARCHAR(50)) PERSISTED;
```

Write advanced T-SQL code

15. Write queries that include regular expressions

REGEXP_LIKE

```
SELECT *
FROM School.Pupils
WHERE REGEXP_LIKE(FirstName, 'c');

SELECT *
FROM School.Pupils
WHERE REGEXP_LIKE(FirstName, 'C');

SELECT *
FROM School.Pupils
WHERE REGEXP_LIKE(FirstName, '[Cc]');

SELECT *
FROM School.Pupils
WHERE REGEXP_LIKE(FirstName, '[O]', 'i');

SELECT *
FROM School.Pupils
WHERE REGEXP_LIKE(FirstName, '^[C-E]');

SELECT *
FROM School.Pupils
WHERE REGEXP_LIKE(FirstName, '[c-e]$', 'i'); -- ends with e.

SELECT *
FROM School.Pupils
WHERE REGEXP_LIKE(FirstName, '[^c-e]$', 'i')

SELECT *
FROM School.Pupils
WHERE REGEXP_LIKE(FirstName, '[A-Za-z]{5}'); -- at least 5 letters.

SELECT *
FROM School.Pupils
WHERE REGEXP_LIKE(FirstName, '^[A-Za-z]{5}$'); -- exactly 5 letters.

SELECT *
FROM School.Pupils
WHERE REGEXP_LIKE(FirstName, '^\w{5}$'); -- exactly 5 letters.
```



```
SELECT *
FROM School.Pupils
WHERE REGEXP_LIKE(FirstName, '^[A-Oa-o]+$'); -- ^…$ is the full string, and
+ is one or more letters

SELECT *
FROM School.Pupils
WHERE REGEXP_LIKE(FirstName, '^(Alice|Daisy)$'); -- Alice or Daisy.

SELECT *
FROM School.Scores
WHERE REGEXP_LIKE(CAST(Score as nchar(3)), '\d8');
```

REGEXP_REPLACE

```
SELECT FirstName, REGEXP_REPLACE(FirstName, '[aeiouAEIOU]', '*') AS
MaskedFirstName
FROM School.Pupils

SELECT FirstName, REGEXP_REPLACE(FirstName, '[aeiouAEIOU]', '*', 2, 2) AS
MaskedFirstName
FROM School.Pupils

SELECT FirstName, REGEXP_REPLACE(FirstName, '[AEIOU]', '*', 2, 2, 'i') AS
MaskedFirstName
FROM School.Pupils

SELECT FirstName, REGEXP_REPLACE(FirstName, '^[A-Za-z]', '*') AS
MaskedFirstName
FROM School.Pupils

SELECT FirstName, REGEXP_REPLACE(FirstName, '[e]$', '(e)') AS
ChangedFirstName
FROM School.Pupils

SELECT FirstName, REGEXP_REPLACE(FirstName, '([e]$)', '\1') AS
ChangedFirstName
FROM School.Pupils

SELECT *, REGEXP_REPLACE(CAST(DateTaken AS NVARCHAR(10)), '(\d{2})(\d{2})-
(\d{2})-([0-9]{2})', '\3.\4.\2') AS ChangedDateTaken
FROM School.Scores

SELECT *, REGEXP_REPLACE(CAST(Score AS varchar(10)), '^(\d+)$', 'Score:
\1') AS ScoreLabel
FROM School.Scores
```

REGEXP_SUBSTR, REGEXP_INSTR and REGEXP_COUNT

```
SELECT * FROM School.Pupils

SELECT FirstName, REGEXP_SUBSTR(FirstName, '[aeiouAEIOU]') AS Letter
```



```
                , REGEXP_INSTR (FirstName, '[aeiouAEIOU]') AS
LetterPosition
                , REGEXP_COUNT (FirstName, '[aeiouAEIOU]') AS LetterCount
FROM School.Pupils

SELECT FirstName, REGEXP_SUBSTR(FirstName, '[aeiouAEIOU]', 2) AS Letter
                , REGEXP_INSTR (FirstName, '[aeiouAEIOU]', 2) AS
LetterPosition
                , REGEXP_COUNT (FirstName, '[aeiouAEIOU]', 2) AS
LetterCount
FROM School.Pupils

SELECT FirstName, REGEXP_SUBSTR(FirstName, '[aeiouAEIOU]', 2, 2) AS Letter
                , REGEXP_INSTR(FirstName, '[aeiouAEIOU]', 2, 2) AS
LetterPosition
FROM School.Pupils

SELECT FirstName, REGEXP_SUBSTR(FirstName, '[AEIOU]', 2, 2, 'i') AS Letter
                , REGEXP_INSTR(FirstName, '[AEIOU]', 2, 2, 0, 'i') AS
LetterPosition1
                , REGEXP_INSTR(FirstName, '[AEIOU]', 2, 2, 1, 'i') AS
LetterPosition2
FROM School.Pupils

SELECT FirstName, REGEXP_SUBSTR(FirstName, '([A-Z])([A-Z])$', 1, 1, 'i', 1)
AS Letter
                , REGEXP_INSTR (FirstName, '([A-Z])([A-Z])$', 1, 1, 0, 'i',
1) AS LetterPosition
FROM School.Pupils

SELECT FirstName, REGEXP_SUBSTR(FirstName, '([A-Z]{2})([A-Z])$', 1, 1, 'i',
1) AS Letter
                , REGEXP_INSTR (FirstName, '([A-Z]{2})([A-Z])$', 1, 1, 0,
'i', 1) AS LetterPosition
FROM School.Pupils

SELECT * FROM School.Scores

SELECT *, REGEXP_SUBSTR(CAST(DateTaken AS NVARCHAR(10)), '(\d{2})(\d{2})-
(\d{2})-(\d{2})', 1, 1, 'i', '3') AS ChangedDateTaken
        , REGEXP_INSTR (CAST(DateTaken AS NVARCHAR(10)), '(\d{2})(\d{2})-
(\d{2})-(\d{2})', 1, 1, 0, 'i', '3') AS DatePosition
        , REGEXP_COUNT (CAST(DateTaken AS NVARCHAR(10)), '\d', 1) AS
NumberOfDigits
FROM School.Scores;

WITH CTE AS (
SELECT CAST(PupilID as varchar(10)) + ' ' + FirstName + ' ' + LastName + '
' + CAST(ClassID as varchar(10)) AS FullString
FROM School.Pupils
)
SELECT FullString, REGEXP_SUBSTR(FullString, '([A-Za-z]+) ([A-Za-z]+)', 1,
1, 'c', 2) AS Name
```



```
                , REGEXP_INSTR (FullString, '([A-Za-z]+) ([A-Za-z]+)', 1,
1, 0, 'i', 2) AS NamePosition
                , REGEXP_COUNT (FullString, '\d', 1) AS NumberOfDigits
                , REGEXP_COUNT (FullString, '\D', 1) AS NumberOfNonDigits
                , REGEXP_COUNT (FullString, '[a-z]', 1, 'i') AS
NumberOfLetters
FROM CTE
```

REGEXP_MATCHES and REGEXP_SPLIT_TO_TABLE

```
SELECT *
FROM REGEXP_MATCHES('Alice Jones', '[A-Z]');

SELECT *
FROM REGEXP_SPLIT_TO_TABLE('Alice Jones', ' ');

SELECT *
FROM REGEXP_MATCHES('Alice Jones', '[A-Z]', 'i');

SELECT *
FROM REGEXP_SPLIT_TO_TABLE('Alice Jones', '[E]', 'i');

SELECT *
FROM REGEXP_MATCHES('Alice Jones', '[A-Z]{2}', 'i');

SELECT FirstName
FROM School.Pupils

SELECT FirstName, M.*
FROM School.Pupils AS P
CROSS APPLY REGEXP_MATCHES(P.FirstName, '[A-Z]{2}', 'i') AS M;

SELECT DateTaken
FROM School.Scores

SELECT DISTINCT DateTaken, M.*
FROM School.Scores AS S
CROSS APPLY REGEXP_SPLIT_TO_TABLE(CAST(S.DateTaken AS VARCHAR(10)), '-') AS
M;
```

16. Fuzzy String Matching functions

EDIT_DISTANCE

```
DECLARE @CharFrom varchar(100) = 'Favourite Programme';
DECLARE @CharTo varchar(100) = 'Favorite Program';

SELECT @CharFrom AS CharFrom,
       @CharTo AS CharTo,
       EDIT_DISTANCE(@CharFrom, @CharTo, 5) AS Distance;
```


EDIT_DISTANCE_SIMILARITY

```
DECLARE @CharFrom varchar(100) = 'Favourite Programme';
DECLARE @CharTo varchar(100) = 'Favorite Program';

SELECT @CharFrom AS CharFrom,
       @CharTo AS CharTo,
       EDIT_DISTANCE(@CharFrom, @CharTo) AS Similarity,
       CASE WHEN LEN(@CharFrom) > LEN(@CharTo) THEN LEN(@CharFrom) ELSE
LEN(@CharTo) END AS MaxLength,
       EDIT_DISTANCE_SIMILARITY(@CharFrom, @CharTo) AS Similarity;

SELECT 100.0 - 100.0 * 3/19 AS Similarity;
```

JARO_WINKLER_DISTANCE

```
DECLARE @CharFrom varchar(100) = 'Favourite Programme';
DECLARE @CharTo varchar(100) = 'Favorite Program';

SELECT @CharFrom AS CharFrom,
       @CharTo AS CharTo,
       JARO_WINKLER_DISTANCE(@CharFrom, @CharTo) AS Similarity,
       JARO_WINKLER_SIMILARITY(@CharFrom, @CharTo) AS Similarity;
```

2e, 17. Graph tables and Write graph queries that use the MATCH operator

```
SELECT * FROM School.Pupils
SELECT * FROM School.PupilFriends

CREATE SCHEMA Graph
GO
-- Node tables
CREATE TABLE Graph.Student (ID INT PRIMARY KEY, name VARCHAR(50)) AS NODE;
CREATE TABLE Graph.Teacher (ID INT PRIMARY KEY, name VARCHAR(50)) AS NODE;

-- Edge table (relationship)
CREATE TABLE Graph.TaughtBy (start_date DATE) AS EDGE;

SELECT * FROM Graph.TaughtBy

-- Students
INSERT INTO Graph.Student VALUES (1, 'Alice');
INSERT INTO Graph.Student VALUES (2, 'Bob');
INSERT INTO Graph.Student VALUES (3, 'Charlie');
INSERT INTO Graph.Student VALUES (4, 'Daisy');
INSERT INTO Graph.Student VALUES (5, 'Ethan');
INSERT INTO Graph.Student VALUES (8, 'Fiona');

-- Teachers
INSERT INTO Graph.Teacher VALUES (1, 'Mr Smith');
INSERT INTO Graph.Teacher VALUES (2, 'Mrs Brown');
INSERT INTO Graph.Teacher VALUES (3, 'Mr Jones');
```



```
SELECT * FROM Graph.Student;
SELECT * FROM Graph.Teacher;

-- Alice taught by Mr Smith
INSERT INTO Graph.TaughtBy VALUES (
    (SELECT $node_id FROM Graph.Student WHERE name = 'Alice'),
    (SELECT $node_id FROM Graph.Teacher WHERE name = 'Mr Smith'),
    '2032-09-01'
);

-- Bob taught by Mr Smith
INSERT INTO Graph.TaughtBy VALUES (
    (SELECT $node_id FROM Graph.Student WHERE name = 'Bob'),
    (SELECT $node_id FROM Graph.Teacher WHERE name = 'Mr Smith'),
    '2032-09-01'
);

-- Charlie taught by Mr Smith
INSERT INTO Graph.TaughtBy VALUES (
    (SELECT $node_id FROM Graph.Student WHERE name = 'Charlie'),
    (SELECT $node_id FROM Graph.Teacher WHERE name = 'Mr Smith'),
    '2032-09-01'
);

INSERT INTO Graph.TaughtBy VALUES (
    (SELECT $node_id FROM Graph.Student WHERE name = 'Daisy'),
    (SELECT $node_id FROM Graph.Teacher WHERE name = 'Mrs Brown'),
    '2032-09-01'
);

INSERT INTO Graph.TaughtBy VALUES (
    (SELECT $node_id FROM Graph.Student WHERE name = 'Ethan'),
    (SELECT $node_id FROM Graph.Teacher WHERE name = 'Mrs Brown'),
    '2032-09-01'
);

INSERT INTO Graph.TaughtBy VALUES (
    (SELECT $node_id FROM Graph.Student WHERE name = 'Fiona'),
    (SELECT $node_id FROM Graph.Teacher WHERE name = 'Mr Jones'),
    '2032-09-01'
);

SELECT * FROM Graph.TaughtBy;

SELECT t.name AS TeacherName
FROM Graph.Student s, Graph.TaughtBy tb, Graph.Teacher t
WHERE MATCH(s-(tb)->t)
  AND s.name = 'Alice';

SELECT s.name AS StudentName
FROM Graph.Student s, Graph.TaughtBy tb, Graph.Teacher t
WHERE MATCH(s-(tb)->t)
```



```
  AND t.name = 'Mr Smith';

--

CREATE TABLE Graph.FriendOf AS EDGE


INSERT INTO Graph.FriendOf ($from_id, $to_id)
VALUES (
    (SELECT $node_id FROM Graph.Student WHERE name = 'Alice'),
    (SELECT $node_id FROM Graph.Student WHERE name = 'Bob')
), (
    (SELECT $node_id FROM Graph.Student WHERE name = 'Alice'),
    (SELECT $node_id FROM Graph.Student WHERE name = 'Charlie')
), (
    (SELECT $node_id FROM Graph.Student WHERE name = 'Bob'),
    (SELECT $node_id FROM Graph.Student WHERE name = 'Ethan')
), (
    (SELECT $node_id FROM Graph.Student WHERE name = 'Charlie'),
    (SELECT $node_id FROM Graph.Student WHERE name = 'Daisy')
), (
    (SELECT $node_id FROM Graph.Student WHERE name = 'Daisy'),
    (SELECT $node_id FROM Graph.Student WHERE name = 'Ethan')
), (
    (SELECT $node_id FROM Graph.Student WHERE name = 'Ethan'),
    (SELECT $node_id FROM Graph.Student WHERE name = 'Fiona')
);


SELECT s2.name AS FriendName
FROM Graph.Student s1, Graph.FriendOf f, Graph.Student s2
WHERE MATCH(s1-(f)->s2)
  AND s1.name = 'Alice';

SELECT DISTINCT
    s1.name AS StartStudent,
    STRING_AGG(s2.name, ' -> ')
        WITHIN GROUP (GRAPH PATH) AS Path
FROM Graph.Student AS s1,
     Graph.FriendOf FOR PATH AS f,
     Graph.Student FOR PATH AS s2
WHERE MATCH(SHORTEST_PATH(s1(-(f)->s2)+)) AND s1.name = 'Alice'

DROP TABLE Graph.Student
DROP TABLE Graph.Teacher
DROP TABLE Graph.TaughtBy
DROP TABLE Graph.FriendOf
DROP SCHEMA Graph
```


Indexes and partitioning

1d. Design and implement indexes

```
SELECT * FROM School.Pupils
SELECT * FROM School.Scores

SELECT p.FirstName, p.LastName, s.Score, s.SetType
FROM School.Pupils AS p
INNER JOIN School.Scores AS s
    ON p.PupilID = s.PupilID;

CREATE NONCLUSTERED INDEX IX_Scores_PupilID
ON School.Scores (PupilID);

---
SELECT Score, DateTaken
FROM School.Scores
WHERE PupilID = 1
  AND SetType = 'Math'
ORDER BY DateTaken DESC;

CREATE NONCLUSTERED INDEX IX_Scores_PupilID_SetType_DateTaken
ON School.Scores (PupilID, SetType, DateTaken DESC) -- You could use ASC as
well.
INCLUDE (Score);
-- PupilID and SetType are in the WHERE, DakeTaken are in the Order.

---
SELECT PupilID, SetType, DateTaken, Score, Maximum
FROM School.Scores
WHERE PupilID = 1
  AND SetType = 'Science';


CREATE NONCLUSTERED INDEX IX_Scores_PupilID_SetType
ON School.Scores (PupilID, SetType)
INCLUDE (DateTaken, Score, Maximum)
---
SELECT PupilID, FirstName, LastName
FROM School.Pupils
WHERE LastName = 'Jones'
-- ORDER BY LastName, FirstName, PupilID

CREATE NONCLUSTERED INDEX IX_Pupils_LastName_FirstName
ON School.Pupils (LastName, FirstName);
---
SELECT PupilID, SetType
FROM School.Scores
WHERE DateRecorded IS NULL
```



```
CREATE NONCLUSTERED INDEX IX_Scores_DateRecorded_Null
ON School.Scores (PupilID, SetType)
WHERE DateRecorded IS NULL;

ALTER INDEX IDX_Scores_PupilID ON School.Scores DISABLE

DROP INDEX IDX_Scores_PupilID ON School.Scores
```

6. Design and implement partitioning for tables and indexes

```
CREATE PARTITION FUNCTION PFNumbers(INT)
AS RANGE RIGHT FOR VALUES (10, 20, 30)

CREATE PARTITION SCHEME PSNumbers
AS PARTITION PFNumbers
TO ([PRIMARY], [PRIMARY], [PRIMARY], [PRIMARY])

CREATE TABLE School.Numbers
(Number int)
ON PSNumbers(Number)

INSERT INTO School.Numbers
SELECT ROW_NUMBER() OVER(ORDER BY (SELECT NULL))
FROM School.Pupils
CROSS JOIN School.Scores

INSERT INTO School.Numbers
VALUES (NULL)

SELECT *, $PARTITION.PFNumbers(Number) AS PartitionNumber
FROM School.Numbers
ORDER BY Number

SELECT $PARTITION.PFNumbers(Number) AS PartitionNumber, MIN(Number) AS
SmallestNumber, MAX(Number) AS BiggestNumber
FROM School.Numbers
GROUP BY $PARTITION.PFNumbers(Number)

DROP TABLE IF EXISTS School.Numbers
DROP PARTITION SCHEME PSNumbers
DROP PARTITION FUNCTION PFNumbers

ALTER PARTITION SCHEME PSNumbers
NEXT USED [PRIMARY]

ALTER PARTITION FUNCTION PFNumbers()
SPLIT RANGE(40)

ALTER PARTITION FUNCTION PFNumbers()
MERGE RANGE(30)
```



```
SELECT SCHEMA_NAME(t.schema_id) AS SchemaName, t.[name] as TableName,
ps.[name] as PSName
FROM sys.tables AS t
JOIN sys.indexes AS i
    ON t.[object_id] = i.[object_id]
JOIN sys.partition_schemes ps
    ON i.data_space_id = ps.data_space_id

SELECT * FROM sys.partition_functions
SELECT * FROM sys.partition_range_values

ALTER TABLE School.Numbers
ADD CONSTRAINT PK_Numbers PRIMARY KEY CLUSTERED (Number)
ON PSNumbers(Number);

CREATE NONCLUSTERED INDEX IX_Numbers_Number -- or CLUSTERED
ON School.Numbers (Number)
ON PSNumbers(Number);
```

1e. Design and implement column store indexes

```
CREATE NONCLUSTERED COLUMNSTORE INDEX IX_ScoresCopy
ON School.ScoresCopy (PupilID)
ORDER (PupilID)
WITH (DATA_COMPRESSION = COLUMNSTORE_ARCHIVE)
ON

ALTER INDEX IX_ScoresCopy
ON School.ScoresCopy REORGANIZE

DROP INDEX IX_ScoresCopy ON School.ScoresCopy
```

Implementing specialised tables

2a. In-memory tables

```
CREATE SCHEMA School
GO

CREATE TABLE [School].[PupilsInMemory](
      [PupilID] [int] PRIMARY KEY NONCLUSTERED NOT NULL,
      [FirstName] [varchar](50) NULL,
      [LastName] [varchar](50) NULL,
      [ClassID] [int] NULL)
WITH (MEMORY_OPTIMIZED=ON, DURABILITY = SCHEMA_AND_DATA)
GO

DROP TABLE School.PupilsInMemory

ALTER DATABASE [DP-800]
SET DELAYED_DURABILITY = DISABLED
```



```
ALTER TABLE [School].[PupilsInMemory]
ADD INDEX IDX_PupilsInMemory_PupilID
HASH (PupilID) WITH (BUCKET_COUNT = 64)

SELECT *
FROM [School].[PupilsInMemory]
WHERE PupilID = 3

ALTER TABLE [School].[PupilsInMemory]
ADD INDEX IDX_PupilsInMemory_PupilID2 (PupilID)
```

2b. Temporal tables

```
CREATE TABLE School.PupilsTemporial (
      [PupilID] [int] PRIMARY KEY NOT NULL,
      [FirstName] [varchar](50) NULL,
      [LastName] [varchar](50) NULL,
      [ClassID] [int] NULL,
    [ValidFrom] DATETIME2 GENERATED ALWAYS AS ROW START,
    [ValidTo] DATETIME2 GENERATED ALWAYS AS ROW END,
    PERIOD FOR SYSTEM_TIME (ValidFrom, ValidTo)
)
WITH (SYSTEM_VERSIONING = ON);

INSERT INTO School.PupilsTemporial(PupilID, FirstName, Lastname, ClassID)
SELECT PupilID, FirstName, Lastname, ClassID
FROM School.Pupils

SELECT *
FROM School.PupilsTemporial

SELECT *
FROM School.PupilsTemporial
FOR SYSTEM_TIME
--BETWEEN '2026-06-08 13:59' AND '2026-06-08 13:59:59'
--FROM '2026-06-08 13:59' TO '2026-06-08 13:59:59'
CONTAINED IN ('2026-06-08 13:50', '2026-06-08 13:59:59')
--ALL

SELECT *
FROM School.PupilsTemporial
FOR SYSTEM_TIME
AS OF '2026-06-08 14:02'

INSERT INTO School.PupilsTemporial(PupilID, FirstName, Lastname, ClassID)
VALUES (9, 'Lizzy', 'Mycroft', 4)

UPDATE School.PupilsTemporial
SET FirstName = 'Georgiana', LastName = 'Smith'
WHERE PupilID = 8
```



```
DELETE School.PupilsTemporial
WHERE PupilID = 5

ALTER TABLE School.PupilsTemporial SET (SYSTEM_VERSIONING = OFF);
DROP TABLE School.PupilsTemporial

CREATE TABLE School.PupilsTemporialHistory
(     [PupilID] [int] NOT NULL,
      [FirstName] [varchar](50) NULL,
      [LastName] [varchar](50) NULL,
      [ClassID] [int] NULL,
    [ValidFrom] DATETIME2 NOT NULL,
    [ValidTo] DATETIME2 NOT NULL)

CREATE TABLE School.PupilsTemporial (
      [PupilID] [int] PRIMARY KEY NOT NULL,
      [FirstName] [varchar](50) NULL,
      [LastName] [varchar](50) NULL,
      [ClassID] [int] NULL,
    [ValidFrom] DATETIME2 GENERATED ALWAYS AS ROW START,
    [ValidTo] DATETIME2 GENERATED ALWAYS AS ROW END,
    PERIOD FOR SYSTEM_TIME (ValidFrom, ValidTo)
)
WITH (SYSTEM_VERSIONING = ON (HISTORY_TABLE =
School.PupilsTemporialHistory));

SELECT *
FROM School.PupilsTemporialHistory

DROP TABLE School.PupilsTemporialHistory
```

2c. External tables

```
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'StrongP@wword123!';

CREATE DATABASE SCOPED CREDENTIAL AzureStorageCredential
WITH
    IDENTITY = 'SHARED ACCESS SIGNATURE',
    SECRET = 'sp=r&st=2026-06-10T07:04:30Z&se=2026-06-
10T15:19:30Z&spr=https&sv=2026-02-
06&sr=b&sig=RUeRSSmOK9easfjFURt%2BIPUD8teAPkg40faiYKL%2Fmko%3D';
GO

CREATE EXTERNAL DATA SOURCE AzureBlobSource
WITH
(
    LOCATION   = 'abs://dp800@dp800storage260610.blob.core.windows.net',
    CREDENTIAL = AzureStorageCredential
);
GO

CREATE EXTERNAL FILE FORMAT CsvFileFormat
WITH
```



```
(
    FORMAT_TYPE = DELIMITEDTEXT,
    FORMAT_OPTIONS
    (
        FIELD_TERMINATOR = ',',
        STRING_DELIMITER = '"',
        FIRST_ROW = 2
    )
);
GO

CREATE EXTERNAL TABLE School.PupilsExternal
(
    PupilID INT,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    ClassID INT
)
WITH
(
    LOCATION = '/PupilData.csv',
    DATA_SOURCE = AzureBlobSource,
    FILE_FORMAT = CsvFileFormat
);
GO

SELECT *
FROM School.PupilsExternal
UNION
SELECT *
FROM School.Pupils
ORDER BY PupilID
GO

DROP EXTERNAL TABLE School.PupilsExternal
DROP EXTERNAL FILE FORMAT CsvFileFormat
DROP EXTERNAL DATA SOURCE AzureBlobSource
DROP DATABASE SCOPED CREDENTIAL AzureStorageCredential
```

2d. Ledger tables

```
CREATE TABLE School.PupilsLedger
(PupilID int NOT NULL,
 FirstName varchar(50) NULL,
 LastName varchar(50) NULL,
 ClassID int NULL)
--WITH (LEDGER = ON (APPEND_ONLY = ON))
WITH ( SYSTEM_VERSIONING = ON (HISTORY_TABLE = School.PupilsLedgerHistory),
LEDGER = ON );

INSERT INTO School.PupilsLedger
SELECT PupilID, FirstName, LastName, ClassID
FROM School.Pupils
```



```
SELECT *, ledger_start_transaction_id, ledger_start_sequence_number
FROM School.PupilsLedger

INSERT INTO School.PupilsLedger(PupilID, FirstName, Lastname, ClassID)
VALUES (9, 'Lizzy', 'Mycroft', 4)

UPDATE School.PupilsLedger
SET FirstName = 'Georgiana', LastName = 'Smith'
WHERE PupilID = 8

DELETE School.PupilsLedger
WHERE PupilID = 5

EXEC sp_generate_database_ledger_digest

DROP TABLE School.PupilsLedger

SELECT * FROM School.PupilsLedgerHistory
```

Implement data security and compliance

25. Design and implement data encryption, including Always Encrypted and column-level encryption

```
SELECT *
INTO dbo.tblReviews2 -- Creates new table.
FROM dbo.tblReviews

SELECT * FROM dbo.tblReviews
SELECT * FROM dbo.tblReviews2

--After re-logging in.
DECLARE @ReviewToFind varchar(1000) = 'Notebooks';
SELECT * FROM dbo.tblReviews2
WHERE Category = @ReviewToFind
```

29. Implement secure database access, including passwordless

Azure SQL Database

```
CREATE USER [PowerPlatformPlan@Filecats.onmicrosoft.com] FROM EXTERNAL
PROVIDER;
```

Fabric SQL Database

```
-- Add Susan as a Viewer
SELECT * FROM tblReviews
-- Remove PowerPlatformPlan as a Viewer, and add user separately.
CREATE USER [Susan@Filecats.onmicrosoft.com] FROM EXTERNAL PROVIDER;
```


28. Design and implement object-level permissions

Azure SQL Database – SQL Server authentication

```
-- My user has permissions to view the Customer table.
SELECT * FROM [SalesLT].[Customer];

-- Create a new user
CREATE USER SalesUser WITH PASSWORD = 'StrongP@ssw0rd!';

-- Can this new user see the Customer table?
EXECUTE AS USER = 'SalesUser';
SELECT * FROM [SalesLT].[Customer];
-- SELECT CustomerID, CompanyName, SalesPerson FROM [SalesLT].[Customer];
SELECT * FROM fn_my_permissions('SalesLT.Customer', 'OBJECT');
REVERT;

-- Grant permissions
GRANT SELECT ON OBJECT::[SalesLT].[Customer] TO SalesUser;

REVOKE SELECT ON OBJECT::[SalesLT].[Customer] TO SalesUser;

GRANT SELECT(CustomerID, CompanyName, SalesPerson) ON
OBJECT::[SalesLT].[Customer] TO SalesUser;

REVOKE SELECT(CompanyName) ON OBJECT::[SalesLT].[Customer] TO SalesUser;

DENY SELECT(CompanyName) ON OBJECT::[SalesLT].[Customer] TO SalesUser;
GRANT SELECT(CompanyName) ON OBJECT::[SalesLT].[Customer] TO SalesUser;

-- Grant vs DENY. Add SalesUser to a role, which has a DENY on the Customer
table
REVOKE SELECT(CustomerID, CompanyName, SalesPerson) ON
OBJECT::[SalesLT].[Customer] TO SalesUser;
GRANT SELECT ON OBJECT::[SalesLT].[Customer] TO SalesUser;

CREATE ROLE NoCustomerAccess;
DENY SELECT ON OBJECT::SalesLT.Customer TO NoCustomerAccess;
ALTER ROLE NoCustomerAccess ADD MEMBER SalesUser;

REVOKE SELECT ON OBJECT::SalesLT.Customer TO NoCustomerAccess;

DROP ROLE NoCustomerAccess;
```

Azure SQL Database – Microsoft Entra MFA authentication

```
-- In PowerPlatformPlan's login:
SELECT ID, Category FROM tblReviews;

-- In admin login:
GRANT SELECT ON SalesLT.Address(AddressID, City) TO
[PowerPlatformPlan@Filecats.onmicrosoft.com];
```



```
-- In PowerPlatformPlan's login:
SELECT ID, Category FROM tblReviews;

-- In admin login:
DENY SELECT ON SalesLT.Address TO
[PowerPlatformPlan@Filecats.onmicrosoft.com];]

-- In PowerPlatformPlan's login:
SELECT ID, Category FROM tblReviews;
```

Fabric SQL Database

```
-- In admin login:
SELECT *
FROM SalesLT.Address;

-- In Susan's login:
SELECT * FROM SalesLT.Address;

-- In admin login:
CREATE USER [Susan@Filecats.onmicrosoft.com] FROM EXTERNAL PROVIDER;

GRANT SELECT ON SalesLT.Address(AddressID, City) TO
[Susan@Filecats.onmicrosoft.com];

-- In Susan's login:
SELECT AddressID, City
FROM SalesLT.Address;

-- In admin login:
DENY SELECT ON SalesLT.Address TO [Susan@Filecats.onmicrosoft.com];

GRANT SELECT ON SalesLT.Address(AddressID, City) TO
[Susan@Filecats.onmicrosoft.com];

-- In Susan's login:
SELECT AddressID, City
FROM SalesLT.Address;

-- After creating a SalesLT role and adding Susan to that role, in Susan's
login:
SELECT *
FROM SalesLT.Customer -- this works

SELECT *
FROM SalesLT.Address -- this does not work, because there is a DENY SELECT
on the table for that user.
```

26. Design and implement Dynamic Data Masking

```
-- Create a database user
CREATE USER SalesReader WITH PASSWORD = 'StrongP@ssw0rd!';

-- Grant SELECT on the Customer table
```



```
GRANT SELECT ON OBJECT::[SalesLT].[Customer] TO SalesReader;

-- Create table using Masking
CREATE TABLE [SalesLT].[CustomerNew](
      [CustomerID] [int] IDENTITY(1,1) NOT NULL,
      [NameStyle] nvarchar(10) MASKED WITH (FUNCTION = 'default()') NOT
NULL,
)

DROP TABLE [SalesLT].[CustomerNew];

-- Add Dynamic Data Masking to the EmailAddress, Phone, ModifiedDate, and
CompanyName columns in the Customer table
ALTER TABLE [SalesLT].[Customer]
ALTER COLUMN EmailAddress ADD MASKED WITH (FUNCTION = 'email()');

ALTER TABLE [SalesLT].[Customer]
ALTER COLUMN EmailAddress DROP MASKED;

ALTER TABLE [SalesLT].[Customer]
ALTER COLUMN Phone ADD MASKED WITH (FUNCTION = 'partial(0,"XXXXXXX",4)');

ALTER TABLE [SalesLT].[Customer]
ALTER COLUMN CustomerID ADD MASKED WITH (FUNCTION = 'random(1,10)');

ALTER TABLE [SalesLT].[Customer]
ALTER COLUMN ModifiedDate ADD MASKED WITH (FUNCTION = 'datetime("YDhms")');

ALTER TABLE [SalesLT].[Customer]
ALTER COLUMN CompanyName ADD MASKED WITH (FUNCTION = 'default()');

SELECT * FROM [SalesLT].[Customer];

-- Test Dynamic Data Masking by executing as the SalesReader user
-- The next statement cannot be used in Fabric SQL Database; you would have
to log as a different user.
EXECUTE AS USER = 'SalesReader';

SELECT * FROM [SalesLT].[Customer]
WHERE CustomerID = 1;

REVERT;
```

27. Design and implement Row-Level Security (RLS)

```
SELECT USER_NAME() AS CurrentUser;
SELECT * FROM [SalesLT].[Product];

-- CREATE users
-- Previously done: CREATE USER SalesReader WITH PASSWORD =
'StrongP@ssw0rd!';
CREATE USER Black WITH PASSWORD = 'StrongP@ssw0rd!';
```



```
-- Grant SELECT on the Product table
GRANT SELECT ON OBJECT::[SalesLT].[Product] TO SalesReader;
GRANT SELECT ON OBJECT::[SalesLT].[Product] TO Black;

-- For best practice, create a Security schema (not required)
CREATE SCHEMA Security;
GO

-- Create inline TVF security predicate
CREATE FUNCTION Security.tvf_ProductColorPredicate(@Color nvarchar(15))
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN
    SELECT 1 AS Result
    WHERE USER_NAME() = 'dbo' OR
         (USER_NAME() = 'SalesReader' AND @Color = 'Red') OR
          USER_NAME() = @Color;
GO

-- Create the security policy
CREATE SECURITY POLICY SalesProductColorFilter
ADD FILTER PREDICATE Security.tvf_ProductColorPredicate(Color)
ON SalesLT.Product
WITH (STATE = ON);
GO

DROP VIEW SalesLT.vProductAndDescription;

-- Test Row Level Security
EXECUTE AS USER = 'SalesReader';
SELECT USER_NAME() AS CurrentUser;
SELECT * FROM [SalesLT].[Product];
REVERT;

EXECUTE AS USER = 'Black';
SELECT USER_NAME() AS CurrentUser;
SELECT * FROM [SalesLT].[Product];
REVERT;

SELECT USER_NAME() AS CurrentUser;
SELECT * FROM [SalesLT].[Product];

-- Clean up
DROP FUNCTION Security.tvf_ProductColorPredicate
DROP SECURITY POLICY SalesProductColorFilter
```


Optimize database performance

33. Recommend database configurations

```
-- These are all possible recommendations. It is not recommended that you
run this script without considering whether it is right for your workload.
ALTER DATABASE [DP-800]
SET AUTOMATIC_TUNING (FORCE_LAST_GOOD_PLAN = ON);

ALTER DATABASE SCOPED CONFIGURATION SET MAXDOP = 10;
GO

SELECT * FROM sys.database_scoped_configurations

ALTER DATABASE SCOPED CONFIGURATION SET OPTIMIZE_FOR_AD_HOC_WORKLOADS = ON;
GO

ALTER DATABASE [DP-800] SET COMPATIBILITY_LEVEL = 160
GO

SELECT * FROM sys.databases
```

34. Preserve data integrity and consistency by using transaction isolation levels and concurrency controls

```
-- Session 1
SELECT * FROM [SalesLT].[Address];

-- Session 2
BEGIN TRANSACTION
UPDATE [SalesLT].[Address]
SET City = 'Toronto ON'
where City = 'Toronto'
-- COMMIT TRANSACTION

-- Session 3
BEGIN TRANSACTION
UPDATE [SalesLT].[Address]
SET City = 'Toronto'
where City in ('Toronto ON', 'Toronto')
-- COMMIT TRANSACTION

-- Session 1
SELECT * FROM [SalesLT].[Address] WITH (SNAPSHOT);

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
--
SELECT * FROM sys.dm_tran_locks
WHERE request_session_id = 82;

SELECT session_id, blocking_session_id,
    start_time, status, command,
```



```
    DB_NAME(database_id) as [database],
    wait_type, wait_resource, wait_time,
    open_transaction_count
FROM sys.dm_exec_requests
WHERE blocking_session_id > 0

DBCC USEROPTIONS

SELECT transaction_isolation_level
FROM sys.dm_exec_sessions
WHERE session_id = 82;


ALTER DATABASE [DP-800]
SET ALLOW_SNAPSHOT_ISOLATION ON

ALTER DATABASE [DP-800]
SET READ_COMMITTED_SNAPSHOT ON


SELECT [name], is_read_committed_snapshot_on, snapshot_isolation_state,
snapshot_isolation_state_desc
FROM sys.databases
```

35. Evaluate query performance by using

35a. Query execution plans

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
--
SET SHOWPLAN_XML OFF

-- Query 1 - This uses Clustered Index Scan
SELECT C1.*, C2.Title
FROM SalesLT.Customer C1
CROSS JOIN SalesLT.Customer C2
WHERE C1.MiddleName IS NOT NULL AND C2.Title = 'Ms.';

-- Compare to Query 2 - This uses Table Scan
WITH C1 AS (SELECT *
FROM SalesLT.Customer2
WHERE MiddleName IS NOT NULL),
C2 AS (SELECT *
FROM SalesLT.Customer2
WHERE Title = 'Ms.')
SELECT C1.*, C2.Title
FROM C1 CROSS JOIN C2
-- ORDER BY C1.CustomerID, C2.CustomerID;
-- OPTION (USE HINT('DISABLE_BATCH_MODE_ADAPTIVE_JOINS'));

-- Create new table
SELECT *
INTO SalesLT.Customer2
```



```
FROM SalesLT.Customer;

-- Enable Batch Mode Adapative Joins
ALTER DATABASE SCOPED CONFIGURATION SET BATCH_MODE_ADAPTIVE_JOINS = ON;

-- Turn statistics off, then update, then turn statistics on
ALTER DATABASE [free-sql-db-4531535] SET AUTO_UPDATE_STATISTICS OFF WITH
NO_WAIT;
UPDATE STATISTICS SalesLT.Customer;
ALTER DATABASE [free-sql-db-4531535] SET AUTO_UPDATE_STATISTICS ON;

--
CREATE NONCLUSTERED INDEX [<Name of Missing Index, sysname,>]
ON [SalesLT].[Customer2] ([MiddleName])
INCLUDE
([CustomerID],[NameStyle],[Title],[FirstName],[LastName],[Suffix],[CompanyN
ame],[SalesPerson],[EmailAddress],[Phone],[PasswordHash],[PasswordSalt],[ro
wguid],[ModifiedDate])
GO

--
ALTER DATABASE SCOPED CONFIGURATION SET BATCH_MODE_ADAPTIVE_JOINS = OFF;
GO
```

35b. Evaluate query performance by using dynamic management views (DMVs)

```
-- The DP-800 database
SELECT * FROM sys.dm_db_resource_stats

-- master database
SELECT * FROM sys.resource_stats

-- The DP-800 database
SELECT * FROM sys.dm_elastic_pool_resource_stats

-- master database
SELECT * FROM sys.elastic_pool_resource_stats

-- The DP-800 database
SELECT * FROM sys.dm_exec_sessions
WHERE session_id = 51
select Db_NAME(5)

SELECT *
FROM sys.dm_exec_requests AS req
CROSS APPLY sys.dm_exec_sql_text(req.sql_handle) AS ST
ORDER BY cpu_time DESC;
```


35c. Evaluate query performance by using Query Store

```
ALTER DATABASE [DP-800] SET QUERY_STORE (OPERATION_MODE = READ_ONLY,
MAX_PLANS_PER_QUERY = 300)
GO

SELECT * FROM sys.database_query_store_options

SELECT TOP 100 * FROM sys.query_store_plan
SELECT TOP 100 * FROM sys.query_store_query
SELECT TOP 100 * FROM sys.query_store_query_text
SELECT TOP 100 * FROM sys.query_store_wait_stats

SELECT *
FROM sys.query_store_plan AS qsp
INNER JOIN sys.query_store_query AS qsq
ON qsp.query_id = qsq.query_id
INNER JOIN sys.query_store_query_text AS qsqt
ON qsq.query_text_id = qsqt.query_text_id
INNER JOIN sys.query_store_wait_stats AS qsws
ON qsp.plan_id = qsws.plan_id
-- WHERE qsq.query_text_id BETWEEN 1657 AND 1800
```

Implement CI/CD by using SQL Database Projects

39. Create, build, and validate database models by using SQL Database Projects, including SDK-style models

```
CREATE TABLE [dbo].[tblCustomers]
(
  [Id] INT NOT NULL PRIMARY KEY,
  [Description] NVARCHAR(100) NOT NULL)
```

44. Update an SQL database project and deploy changes

```
CREATE TABLE [dbo].[tblCustomers]
(
  [Id] INT NOT NULL PRIMARY KEY,
  [Description] NVARCHAR(101) NOT NULL)
```

38. Create and manage reference/static data in source control

```
DELETE FROM dbo.tblCustomers
WHERE Id IN (-1, -2);
INSERT INTO dbo.tblCustomers(Id, Description)
VALUES (-1, 'First name'), (-2, 'Second name');
---
MERGE INTO dbo.tblCustomers as target
USING (VALUES (-1, 'First name'), (-2, 'Second name')) AS source (Id,
Description)
ON target.Id = source.Id
WHEN NOT MATCHED THEN
```



```
INSERT (Id, Description)
VALUES (source.Id, source.Description);
```

43. Detect schema drift by using SQL Database Projects

```
-- In the database
ALTER TABLE dbo.tblTest
ADD Notes nvarchar(100) NULL;

ALTER TABLE dbo.tblTest
ALTER COLUMN Description nvarchar(200) NULL;

-- In the project
CREATE TABLE [dbo].[tblTest]
(
  [Id] INT NOT NULL PRIMARY KEY,
  [Description] NVARCHAR(100) NOT NULL,
  [Note] NVARCHAR(100) NULL
) --[Singular "Note"]
```

37. Design and implement a testing strategy, including unit tests and integration tests

```
DECLARE @RC AS INT, @Description AS NVARCHAR (200);
DELETE FROM dbo.tblTest WHERE Id = -555;
SELECT @RC = 0, @Description = 'John Smith';
EXECUTE @RC = [dbo].[procTest] @Description;
SELECT @RC AS RC;
```

41. Manage branching, pull requests, and conflict resolution

```
-- In Dev branch
CREATE TABLE [dbo].[tblTest]
(
  [Id] INT NOT NULL PRIMARY KEY,
  [Description] NVARCHAR(100) NOT NULL,
  [Note] NVARCHAR(100) NULL,
  [IsCurrent] BIT NULL
)

-- In Main branch
CREATE TABLE [dbo].[tblTest]
(
  [Id] INT NOT NULL PRIMARY KEY,
  [Description] NVARCHAR(100) NOT NULL,
  [Note] NVARCHAR(100) NULL,
  [DateEntry] DATETIME NULL,
  [IsCurrent] BIT NULL
)

-- In Dev branch
CREATE TABLE [dbo].[tblTest]
(
```



```
  [Id] INT NOT NULL PRIMARY KEY,
  [Description] NVARCHAR(100) NOT NULL,
  [Note] NVARCHAR(100) NULL,
  [DateStart] DATETIME NULL,
  [IsCurrent] BIT NULL
)
```

42. Implement secrets management

```
      - name: Deploy
        uses: azure/sql-action@v2
        with:
              connection-string: ${{secrets.SQLCONNECTIONSTRING}}
```

Integrate SQL solutions with Azure services

49. Expose database objects – tables

```
dab init --database-type mssql --config dab-config.json --connection-string
"Server=…;Connection Timeout=30;"

dab add Address --source SalesLT.Address --permissions "anonymous:read"

dab add Customer --source SalesLT.Customer --permissions "anonymous:read"

dab add CustomerAndAddress --source SalesLT.vCustomerAndAddress --
permissions "anonymous:read"
```

http://localhost:5000/health
http://localhost:5000/api/Address
http://localhost:5000/swagger

Using GraphQL

```
query AddressList {

  addresses(filter: {or: [{City:{eq: "Bothell"}}, {AddressID:{lte: 100}}]})

  {items

    {ID:AddressID City}}

}
```

46. Create configuration files for Data API builder (DAB)

```
dab init --database-type mssql --config dab-config.json --connection-string
"Server=…;Connection Timeout=30;"
dab add Address --source SalesLT.Address --permissions "anonymous:read‟
dab add Customer --source SalesLT.Customer --permissions "anonymous:read"
dab add CustomerAddress --source SalesLT.CustomerAddress --permissions
"anonymous:read"
dab start
```


47. Configure entities for REST and GraphQL, including data caching, pagination, searching, and filtering

```
http://localhost:5000/api/Address?$filter=AddressID eq 25
http://localhost:5000/api/Address?$filter=AddressID gt 25
http://localhost:5000/api/Address?$first=10
http://localhost:5000/api/Address?$first=10&$after=ABCD
http://localhost:5000/graphql
```

48. Configure REST or GraphQL endpoints

49. stored procedures, and views

```
dab update CustomerAndAddress --source SalesLT.vCustomerAndAddress --
source.key-fields CustomerID --permissions "anonymous:read"

dab add Product700 --source SalesLT.Product700 --source.type "stored-
procedure" --permissions "anonymous:*"

dab add ProductProc --source SalesLT.ProductProc --source.type "stored-
procedure" --permissions "anonymous:*" --parameters.name
"FirstProd,LastProd" --parameters.description "First Product,Last Product"
--parameters.required "true,true" --parameters.default "700,799"
```

49. including GraphQL relationships

```
dab update Customer --relationship CustomerAddresses --target.entity
CustomerAddress --cardinality many --relationship.fields
"CustomerID:CustomerID"
dab update CustomerAddress --relationship Address --target.entity Address -
-cardinality one --relationship.fields "AddressID:AddressID"
-- In http://localhost:5000/graphql/
query CustomersWithAddresses
{
  customers(filter: {CustomerID: {eq: 29485}})
  {
    items
    {
      CustomerID FirstName LastName CustomerAddresses
      {
        items
        {
          AddressType
          Address
          {
            AddressID AddressLine1 AddressLine2 City StateProvince
PostalCode CountryRegion
          }
        }
      }
    }
  }
```



```
}
```

50. Configure and implement DAB deployment

```
winget install --exact --id Microsoft.AzureCLI
az version
az login
az acr build --registry dp800dab.azurecr.io --image dab:latest .
```

46. Create configuration files for Data API builder (DAB)

```
-- In the .env file
devconnectionstring=Server=tcp:dp-800-
260501.database.windows.net,1433;Initial Catalog=DP-800;Persist Security
Info=False;User
ID=sqladmin;Password=SQLStrongPassword123!;MultipleActiveResultSets=False;E
ncrypt=True;TrustServerCertificate=False;Connection Timeout=30;

-- In the dab-config.json file
    "connection-string": "@env('devconnectionstring')"
```

52. Handle changes

See topic 55.

Design and implement models and embeddings

54. Create and manage external models

```
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'StrongP@wword123!';

CREATE DATABASE SCOPED CREDENTIAL [https://dp800foundry.openai.azure.com/]
WITH IDENTITY = 'Managed Identity', SECRET =
'{"resourceid":"https://cognitiveservices.azure.com"}'

CREATE EXTERNAL MODEL EmbeddingModel
WITH (
        LOCATION =
'https://dp800foundry.openai.azure.com/openai/deployments/text-embedding-3-
small/embeddings?api-version=2024-10-21',
        API_FORMAT = 'Azure OpenAI',
        MODEL_TYPE = EMBEDDINGS,
        MODEL = 'text-embedding-3-small',
        CREDENTIAL = [https://dp800foundry.openai.azure.com/]
)

SELECT AI_GENERATE_EMBEDDINGS('This product was great.' USE MODEL
EmbeddingModel)
```

55a. Choose an embedding maintenance method - Table triggers

```
ALTER TRIGGER dbo.trgReviewsUpdate
ON dbo.tblReviews
```



```
AFTER UPDATE
AS
BEGIN
      SELECT ID FROM inserted
      IF UPDATE(Review)
      BEGIN
            UPDATE tblReviews
            SET vctVector = NULL
            WHERE ID IN (SELECT ID FROM inserted)
            EXEC UpdateReviewEmbeddings
      END
END

CREATE TRIGGER dbo.trgReviewsInsert
ON dbo.tblReviews
AFTER INSERT
AS
BEGIN
      EXEC UpdateReviewEmbeddings
END

SELECT * FROM [dbo].[tblReviews]

UPDATE tblReviews
SET Review = 'Three star. very low memory. Not good buy.'
WHERE ID = 10

INSERT INTO dbo.tblReviews (ID, Category, ProductName, Review)
VALUES (106, 'Books', 'Book 6', 'This book is fantastic! I highly recommend
it to everyone.')

DELETE FROM dbo.tblReviews
WHERE ID = 106

UPDATE tblReviews
SET Review = 'two star.Return looking for Mac pro.'
WHERE ID = 10
```

55b, 52c. Change Tracking

```
INSERT INTO [School].[PupilsCopy] (PupilID, FirstName, LastName, ClassID)
VALUES (11, 'John', 'Smith', 1)

UPDATE [School].[PupilsCopy]
SET FirstName = 'John', LastName = 'Burton'
WHERE PupilID = 11

SELECT * FROM [School].[PupilsCopy]

ALTER DATABASE [DP-800]
SET CHANGE_TRACKING = ON
(CHANGE_RETENTION = 2 DAYS, AUTO_CLEANUP = ON)
```



```
SELECT * FROM sys.change_tracking_databases
WHERE database_id = DB_ID('DP-800')

ALTER TABLE School.PupilsCopy
ENABLE CHANGE_TRACKING
WITH (TRACK_COLUMNS_UPDATED = ON)

SELECT * FROM CHANGETABLE(CHANGES School.PupilsCopy, 0) AS CT

DECLARE @last_sync_version bigint
SET @last_sync_version = CHANGE_TRACKING_CURRENT_VERSION()

SELECT @last_sync_version

UPDATE [School].[PupilsCopy]
SET FirstName = 'John', LastName = 'Burton'
WHERE PupilID = 10

SELECT * FROM CHANGETABLE(CHANGES School.PupilsCopy, 0) AS CT
SELECT * FROM CHANGETABLE(CHANGES School.PupilsCopy, @last_sync_version) AS
CT

SELECT * FROM sys.change_tracking_tables
SELECT * FROM sys.objects
WHERE object_id = 903674267
```

55c, 52d. Azure Functions with SQL trigger binding

55d, 52e. Azure Logic Apps

```
ALTER TABLE [School].[PupilsCopy]
ADD RowVer rowversion;

select * from [School].[PupilsCopy]

UPDATE [School].[PupilsCopy]
set PupilID = 10
WHERE PupilID = 9
```

55e, 52b. Change Data Capture (CDC)

```
EXEC sys.sp_cdc_enable_db
GO

SELECT *
FROM [dbo].[systranschemas]

EXEC sys.sp_cdc_enable_table @source_schema = N'School',
                             @source_name = N'PupilsCopy',
                             @role_name = NULL,
                             @supports_net_changes = 1
```



```
select * from [School].[PupilsCopy]
INSERT INTO [School].[PupilsCopy] (PupilID, FirstName, LastName, ClassID)
VALUES (9, 'John', 'Smith', 1)

UPDATE [School].[PupilsCopy]
SET PupilID = 10, FirstName = 'Jane', LastName = 'Burton', ClassID = 2
WHERE PupilID = 9

-- It is not recommended to query this table directly
SELECT * FROM [cdc].[School_PupilsCopy_CT]

-- But to use this function to get the changes
DECLARE @from_lsn binary(10), @to_lsn binary(10)

SET @from_lsn = sys.fn_cdc_get_min_lsn('School_PupilsCopy')
SET @to_lsn = sys.fn_cdc_get_max_lsn()

SELECT * FROM cdc.fn_cdc_get_all_changes_School_PupilsCopy (@from_lsn,
@to_lsn, 'all')

DELETE FROM [School].[PupilsCopy]
WHERE PupilID = 10
```

55f, 52a. Change Event Streaming (CES)

```
CREATE DATABASE SCOPED CREDENTIAL DP800Credential
    WITH IDENTITY = 'DP800',
    SECRET = '<Primary_Key>'

EXEC sys.sp_enable_event_stream

EXEC sys.sp_create_event_stream_group
    @stream_group_name =      N'DP800StreamGroupName',
    @destination_type =       N'AzureEventHubsAmqp',
    @destination_location =   N'<Name>.servicebus.windows.net/dp800',
    @destination_credential = DP800Credential,
    @max_message_size_kb =    1024,
    @partition_key_scheme =   N'None'

EXEC sys.sp_add_object_to_event_stream_group
    N'DP800StreamGroupName',
    N'School.PupilsCopy'

    exec sys.sp_cdc_disable_db

select * from sys.databases
```

55g. Microsoft Foundry

57. Design and implement chunks for embeddings

```
-- Using single string
Declare @SingleReview AS varchar(8000)
SELECT @SingleReview = Review
FROM tblReviews
WHERE ID = 106;

SELECT @SingleReview, LEN(@SingleReview)

SELECT * FROM ai_generate_chunks(SOURCE = @SingleReview, CHUNK_TYPE =
FIXED, CHUNK_SIZE = 500)

-- Using table data
SELECT * FROM tblReviews CROSS APPLY
              AI_GENERATE_CHUNKS(Source = Review, CHUNK_TYPE = FIXED,
CHUNK_SIZE = 500, OVERLAP = 25, ENABLE_CHUNK_SET_ID = 1)
ORDER BY ID DESC

-- Deleting this review
DELETE FROM tblReviews
WHERE ID = 106;
```

58. Generate embeddings

```
DECLARE @StartNumber INT = 1
WHILE @StartNumber < 200
BEGIN
      SELECT *, AI_GENERATE_EMBEDDINGS(Review USE MODEL EmbeddingModel) AS
Embedding
      FROM tblReviews
      WHERE ID between @StartNumber and @StartNumber+19
      SET @StartNumber = @StartNumber + 20
      WAITFOR DELAY '00:00:04'
END
```

Design and implement intelligent search

61. Design for vector data, including vector data type, vector indexes, and size

```
ALTER TABLE tblReviews ADD vctVector VECTOR(1536);
ALTER DATABASE SCOPED CONFIGURATION
SET PREVIEW_FEATURES = ON;
GO
ALTER TABLE tblReviews ADD vctVector2 VECTOR(1536, float16);
ALTER TABLE tblReviews DROP COLUMN vctVector2;
```

65. Implement vector search

```
CREATE PROC UpdateReviewEmbeddings AS
BEGIN
```



```
	DECLARE @NumberOfRowsUpdated INT = 999;
	WHILE @NumberOfRowsUpdated > 0
	BEGIN

		UPDATE TOP (10) tblReviews
		SET vctVector = AI_GENERATE_EMBEDDINGS(Review USE MODEL
EmbeddingModel)
		FROM tblReviews
		WHERE vctVector IS NULL AND Review IS NOT NULL;
		SET @NumberOfRowsUpdated = @@ROWCOUNT
		WAITFOR DELAY '00:00:04'
	END
END
```

62. Identify when to use vector-related types and functions for semantic searching, including

VECTOR_NORMALIZE

```
SELECT
    ID,
    Category,
    ProductName,
    vctVector,
    VECTOR_NORMALIZE(vctVector, 'norm2') AS NormalizedVector
FROM dbo.tblReviews
WHERE vctVector IS NOT NULL;
```

VECTORPROPERTY

```
DECLARE @vctTarget vector(1536);

SELECT @vctTarget = vctVector
FROM dbo.tblReviews
WHERE ID = 1;

SELECT VECTORPROPERTY(@vctTarget, 'Dimensions') AS TargetVectorProperties;
SELECT VECTORPROPERTY(@vctTarget, 'BaseType') AS TargetVectorProperties;
```

VECTOR_DISTANCE

```
DECLARE @vctTarget vector(1536);

SELECT @vctTarget = vctVector
FROM dbo.tblReviews
WHERE ID = 1;

SELECT TOP 10
    ID,
    Category,
    ProductName,
    vctVector,
    VECTOR_DISTANCE('cosine',vctVector, @vctTarget) AS Distance
```



```
FROM dbo.tblReviews
WHERE vctVector IS NOT NULL
ORDER BY Distance;
```

VECTOR_SEARCH

```
DECLARE @vctTarget vector(1536);

SELECT @vctTarget = vctVector
FROM tblReviews
WHERE ID =1;

SELECT TOP(10) WITH APPROXIMATE *
FROM VECTOR_SEARCH(
       TABLE = dbo.tblReviews,
       COLUMN = vctVector,
       SIMILAR_TO = @vctTarget,
       METRIC = 'cosine') AS SearchResults
ORDER BY Distance
```

64. Evaluate vector index types and metrics

```
--ENN, kNN VECTOR_DISTANCE
--ANN      VECTOR_SEARCH

ALTER DATABASE SCOPED CONFIGURATION
SET PREVIEW_FEATURES = ON;
GO

CREATE VECTOR INDEX idxVector ON tblReviews(vctVector) WITH
(METRIC='cosine', TYPE = 'DiskAnn')
```

60. Implement full-text search

Creating Fulltext Catalog and Index

```
CREATE FULLTEXT CATALOG catReviews WITH ACCENT_SENSITIVITY = ON AS DEFAULT;
--ALTER FULLTEXT CATALOG catReviews AS DEFAULT

SELECT * FROM sys.fulltext_catalogs;

CREATE FULLTEXT INDEX ON tblReviews(Review LANGUAGE 1033) KEY INDEX
[PK_tblReviews]

SELECT * FROM sys.fulltext_indexes
```

Using the Fulltext index

```
SELECT * FROM tblReviews WHERE CONTAINS(Review, 'sound')
SELECT * FROM tblReviews WHERE CONTAINS(Review, 'sound OR MAC')
SELECT * FROM tblReviews WHERE CONTAINS(Review, '"three star"')
SELECT * FROM tblReviews WHERE CONTAINS(Review, '"three star*"')

SELECT * FROM tblReviews
WHERE CONTAINS(Review, 'FORMSOF(INFLECTIONAL, "worked")')
```



```
SELECT * FROM tblReviews WHERE CONTAINS(Review, 'NEAR((not, working), 1)')
SELECT * FROM tblReviews WHERE CONTAINS(Review, 'NEAR((not, working), 3)')
SELECT * FROM CONTAINSTABLE(tblReviews, Review, 'star');

SELECT * FROM CONTAINSTABLE(tblReviews, Review, 'star') AS CT
INNER JOIN tblReviews R ON R.ID = CT.[KEY];

SELECT * FROM CONTAINSTABLE(tblReviews, Review, 'star', 3) AS CT
INNER JOIN tblReviews R ON R.ID = CT.[KEY];

SELECT * FROM tblReviews
WHERE FREETEXT(Review, 'second hand')

SELECT * FROM tblReviews
WHERE FREETEXT(Review, 'price too high')

SELECT * FROM FREETEXTTABLE(tblReviews, Review, 'price too high', 4) AS CT
INNER JOIN tblReviews R ON R.ID = CT.[KEY];
```

67. Implement reciprocal rank fusion (RRF)

```
DROP TABLE IF EXISTS #tblKeyword;
DROP TABLE IF EXISTS #tblSemantic;
DECLARE @SearchTerm NVARCHAR(1000) = 'good review';

-- Fulltext search
SELECT TOP 15 id, RANK() OVER (ORDER BY FTRank DESC) AS rank
INTO #tblKeyword
FROM
  (
    SELECT TOP 15 id, Category, ProductName, ftt.[RANK] AS FTRank
    FROM tblReviews R
    INNER JOIN FREETEXTTABLE(tblReviews, *, @SearchTerm) AS ftt ON R.id =
ftt.[KEY]
    ORDER BY FTRank DESC
  ) AS FreetextDocuments
ORDER BY rank ASC

SELECT * FROM #tblKeyword;

-- Vector search
DECLARE @vctTarget vector(1536) = AI_GENERATE_EMBEDDINGS(@SearchTerm USE
MODEL EmbeddingModel);

With tblSemantic AS (
SELECT TOP(15) WITH APPROXIMATE ID, distance
FROM VECTOR_SEARCH(
       TABLE = dbo.tblReviews,
       COLUMN = vctVector,
       SIMILAR_TO = @vctTarget,
       METRIC = 'cosine') AS SearchResults
ORDER BY Distance)
```



```
SELECT ID, RANK() OVER (ORDER BY Distance) AS rank
INTO #tblSemantic
FROM tblSemantic

SELECT * FROM #tblSemantic;

-- Reciprocal Rank Fusion (RRF)
WITH tblRRF AS
(SELECT TOP 15
  COALESCE(ss.id, ks.id) AS id,
  ss.rank AS SemanticRank,
  ks.rank AS KeywordRank,
  COALESCE(1.0 / (60 + ss.rank), 0.0) +
  COALESCE(1.0 / (60 + ks.rank), 0.0) AS score
FROM #tblSemantic ss
FULL OUTER JOIN #tblKeyword ks ON ss.id = ks.id
ORDER BY score DESC)
SELECT RRF.*, Category, ProductName, Review
FROM tblRRF AS RRF
INNER JOIN dbo.tblReviews AS R
ON RRF.ID = R.ID
```

69-73. Design and implement retrieval-augmented generation (RAG)

```
DECLARE @Payload NVARCHAR(MAX);
DECLARE @Response NVARCHAR(MAX);
DECLARE @strQuestion NVARCHAR(MAX) = 'What is a good laptop for me to use
for work?';
DECLARE @vctQuestion VECTOR(1536);
DECLARE @context NVARCHAR(MAX);

SET @vctQuestion = AI_GENERATE_EMBEDDINGS(@strQuestion USE MODEL
EmbeddingModel)

-- Retrieve closest review based on cosine similarity
SET @context = (
                SELECT TOP(5) WITH APPROXIMATE ID, Category, ProductName,
Review
                FROM VECTOR_SEARCH(
                                TABLE = dbo.tblReviews,
                                COLUMN = vctVector,
                                SIMILAR_TO = @vctQuestion,
                                METRIC = 'cosine')
                ORDER BY distance
                FOR JSON PATH
                )

SELECT @context as Context;

-- Build prompt
```



```
SET @payload = JSON_OBJECT(
                          'messages': JSON_ARRAY(
                          JSON_OBJECT(
                                     'role': 'system',
                                     'content': 'You are a helpful
assistant that answers questions about products.'
                          ), JSON_OBJECT(
                                     'role': 'user',
                                     'content': 'Product reviews:' +
ISNULL(@context, '[]')
                                                 + CHAR(10) + CHAR(10) +
                                                'Customer question: ' +
@strQuestion)));

SELECT @payload as PayLoad;

-- Call ChatGPT (Azure OpenAI) endpoint
EXEC sys.sp_invoke_external_rest_endpoint
    @url = N'https://dp800foundry.openai.azure.com/openai/deployments/gpt-
5.4-mini/chat/completions?api-version=2024-10-21',
    @method = N'POST',
    @payload = @payload,
    @credential = [https://dp800foundry.openai.azure.com/],
    @response = @Response OUTPUT;

SELECT @Response AS ChatGPT_Response; -- For debugging purposes

SELECT JSON_VALUE(@response, '$.result.choices[0].message.content') AS
Answer;
```

52. Handle changes by using change event streaming (CES), change data capture (CDC), Change Tracking, Azure Functions with SQL trigger binding, or Azure Logic Apps

Change Event Streaming (CES)

```
CREATE MASTER KEY ENCRYPTION BY PASSWORD = '<Password>';

CREATE DATABASE SCOPED CREDENTIAL <CredentialName>
    WITH IDENTITY = 'SHARED ACCESS SIGNATURE',
    SECRET = '<Generated SAS Token>'
-- or WITH IDENTITY = 'Managed Identity'
-- or WITH IDENTITY = '<Azure Event Hubs SAS Policy name>', SECRET =
'<Primary or Secondary key value>'
```



```
EXEC sys.sp_enable_event_stream

EXEC sys.sp_create_event_stream_group
    @stream_group_name =      N'<EventStreamGroupName>',
    @destination_type =       N'AzureEventHubsAmqp',
    @destination_location =
N'<AzureEventHubsHostName>/<EventHubsInstance>',
    @destination_credential = <CredentialName>,
    @max_message_size_kb =    <MaxMessageSize>,
    @partition_key_scheme =   N'<PartitionKeyScheme>'

EXEC sys.sp_add_object_to_event_stream_group
    N'<EventStreamGroupName>',
    N'<SchemaName>.<TableName>'

SELECT name, is_event_stream_enabled FROM sys.databases;
```

Change Data Capture (CDC)

```
SELECT is_cdc_enabled, *
FROM sys.databases;

EXEC sys.sp_cdc_enable_db;
GO

select * from [cdc].[captured_columns]

EXEC sys.sp_cdc_enable_table
    @source_schema = N'dbo',
    @source_name = N'tblReviews',
    @role_name = NULL;

UPDATE dbo.tblReviews
SET ID = 200
WHERE ID = 100;

SELECT * FROM [cdc].[dbo_tblReviews_CT]


DECLARE @from_lsn binary(10);
DECLARE @to_lsn   binary(10);

SET @from_lsn = sys.fn_cdc_get_min_lsn('dbo_tblReviews');
SET @to_lsn   = sys.fn_cdc_get_max_lsn();

SELECT *
FROM cdc.fn_cdc_get_all_changes_dbo_tblReviews(@from_lsn, @to_lsn, 'all');
```

Change Tracking

```
ALTER DATABASE [free-sql-db-4531535]
SET CHANGE_TRACKING = ON
(CHANGE_RETENTION = 3 DAYS, AUTO_CLEANUP = ON)
```



```
ALTER TABLE [dbo].[tblReviews]
ENABLE CHANGE_TRACKING
WITH (TRACK_COLUMNS_UPDATED = ON)

SELECT *
FROM sys.change_tracking_databases;

SELECT *
FROM sys.change_tracking_tables;

declare @synchronization_version bigint;

-- Obtain the current synchronization version. This will be used next time
that changes are obtained.
SET @synchronization_version = CHANGE_TRACKING_CURRENT_VERSION();

UPDATE dbo.tblReviews
SET ID = 100
WHERE ID = 200;

declare @last_synchronization_version bigint;

SELECT
    CT.* --, CT.SYS_CHANGE_OPERATION,
    -- CT.SYS_CHANGE_COLUMNS, CT.SYS_CHANGE_CONTEXT
FROM
    CHANGETABLE(CHANGES tblReviews, @last_synchronization_version) AS CT;

UPDATE dbo.tblReviews
SET Category = 'Ink jet printers'
WHERE Category = 'Inkjet printers';
```

Azure Logic Apps

```
ALTER TABLE dbo.tblReviews
ADD RowVer rowversion;

select * from dbo.tblReviews;
```

